UNIVERSITY OF GHANA

DEPARTMENT OF COMPUTER SCIENCE

DCIT 204/308: DATA STRUCTURES AND ALGORITHMS I & II

UG CAMPUS SECURITY & EMERGENCY RESPONSE OPTIMIZER

Technical Report

PROJECT INFORMATION

University of Ghana, Legon campus security and emergency-response operations

Campus security and emergency-response dispatch, routing, scheduling and optimisation

Java 17 | SQLite | Benchmark seed 7497

681 tests passed; 0 failures; 0 errors; 0 skipped

Group 16 - Black Box | tkay0/ug-security-optimizer

TEAM MEMBERS AND INDEX NUMBERS

1. Isaac Morrison Quaye - 22079872
2. Eastwood Tweneboah Osei - 22040957
3. Selorm Sem - 22243032
4. Fauziya Adjeley Adjei - 22153370
5. Nana Kwasi Agyiri - 22136496
6. Jephthah Peprah - 22036173
7. Abukari Issah Kangre - 22404379
8. Owusu Kenneth Kwabena - 22401243
9. Asiedu Messiah - 22392636
10. Kumi Kwame Esua - 22396052
11. Owusu-Ansah John Nana Kwaku Bonsu - 22409852
12. Amoh Derrick Kwaku - 22367855
13. Adjapong Stephanie Kyerewaa - 22381560
14. Jimoh Damilola Alliyah - 22325573
15. Awuku Duke Asare - 22400447

Executive Summary

The UG Campus Security & Emergency Response Optimizer is a Java 17 academic system designed around a University of Ghana, Legon context. The system stores localized operational data in SQLite, loads and processes that data through custom-built data structures, and applies searching, sorting, graph, greedy, dynamic-programming and exhaustive algorithms to realistic campus-security and emergency-response tasks.

The completed implementation integrates a two-mode Swing desktop interface, persistent database services, routing and resource assignment, correctness evidence, and a deterministic efficiency laboratory. Operational Mode presents Dashboard, Incidents, Dispatch, Routes, Resources and Reports in operator-facing language, while Academic / DSA Lab exposes Structures, Search & Sort, Graph Algorithms, Optimization, Correctness and Efficiency Lab for assessment. Both modes share the same frontend service adapters, application services, SQLite state and custom DSA/algorithm implementations. The two-mode interface was merged to the final main branch in PR #40 (merge commit ff7f47a), and the verified suite contains 681 passing JUnit tests with a successful Maven build and package step. The software, correctness, performance, and report requirements are complete; remaining assessment activities are the required demonstration video, oral defence, and preservation of supporting attendance evidence.

The final performance experiment was executed with benchmark seed 7497 and produced 336 genuine raw trial rows. Each official benchmark group uses three measured trials and reports the group average. Results support the expected scalability differences between linear and binary search, quadratic and n log n sorting, unbalanced and balanced search trees, heap-based priority dispatch, graph algorithms, and dynamic programming versus brute force. Measured timings are interpreted as empirical application-level evidence rather than microbenchmark-grade guarantees because JVM warm-up, garbage collection, cache effects and operating-system scheduling can influence short runs.

Table of Contents

1. Problem Statement, Assumptions, Inputs/Outputs and System Boundaries

1.1 Local Problem Context

University campuses contain many interconnected locations and depend on limited personnel and emergency resources. When an incident is reported, an operator may need to decide which request should be handled first, which compatible resource is available, what route is suitable under current road conditions, and how the decision should be recorded for later review. The project adapts the official Ghana Smart Service Operations Optimizer brief to the University of Ghana, Legon campus-security and emergency-response context.

1.2 Core Computational Problems

1. Prioritise pending service requests under FIFO, urgency-based and priority-heap rules.
2. Search and sort operational records using custom algorithm implementations.
3. Determine reachable locations and traversal order using BFS and DFS.
4. Find a lowest-cost route between campus locations using Dijkstra on a weighted road graph.
5. Build minimum spanning structures using Prim and Kruskal for network-analysis demonstrations.
6. Assign an eligible emergency/security resource using a greedy decision process.
7. Select an optimal subset of incidents under a capacity/budget using dynamic programming, with brute force as a small-n exact baseline.
8. Persist operational changes, audit events and algorithm evidence so the system can be rerun and defended.

1.3 Inputs

- Localized campus locations, roads, service requests and resources loaded from validated CSV seed files into SQLite.
- Operator input from Operational Mode for incident reporting, dispatch, routing, resources and reports, plus examiner actions from Academic / DSA Lab for structures, algorithms, correctness traces and efficiency experiments.
- Index-derived project parameters: priority weight 3, optimization budget 80 and deterministic benchmark seed 7497.
- Road-condition and scenario information used to construct weighted routing graphs.

1.4 Outputs

- Prioritized request order and scheduling traces.
- Search/sort results and tree-index demonstrations.
- BFS/DFS traversal output, Dijkstra path/cost output, and Prim/Kruskal MST evidence.
- Resource recommendations and persisted assignments with workflow/audit history.
- DP/brute-force optimization results and comparison evidence.
- Raw benchmark CSV, averaged benchmark summary, environment metadata, graphs and correctness traces.

1.5 Assumptions and Boundaries

- The project is an academic simulation; records and coordinates are synthetic/schematic rather than a live University of Ghana emergency-management system.
- Routing cost is derived from the stored weighted-road data; the system does not claim live traffic or GPS integration.
- Benchmark memory values are approximate JVM used-heap deltas and are not exact theoretical space-complexity measurements.
- The Swing interface separates operator-facing workflows from examiner-facing DSA evidence while remaining intentionally lightweight; the official assessment emphasizes data structures, algorithms, correctness, database integration and empirical analysis rather than production-grade UI design.

1.6 Five Major Operation Specifications

Table 1. Core operations: inputs, outputs and operating conditions

Operation	Input	Output	Important conditions
Priority dispatch	Pending requests	Next request / priority order	Uses index-derived priority weight and custom heap
Shortest route	Source, destination, road/scenario graph	Path and weighted cost	Dijkstra requires non-negative edge costs
Resource assignment	Request + eligible resources + route costs	Recommended/persisted assignment	Greedy choice only considers compatible reachable resources
Budget optimization	Candidate incidents + budget 80	Selected optimal subset	DP and brute force solve the same exact objective for comparison workloads
Efficiency experiment	Algorithm family + input-size plan + seed 7497	Raw trials, averages, memory estimate, result metric	Three measured trials per group; setup excluded from timed region where practical

2. Dataset Description, Data Dictionary and Database Schema

2.1 Dataset Composition

The canonical dataset is localized to the University of Ghana, Legon context. It contains campus locations, weighted road connections, service requests and resources. The final running application imports the canonical CSV data into SQLite when the database is empty and then reads/writes persistent records through DAO classes.

Table 2. Dataset record counts against the official minimums

Entity	Official minimum	Project evidence	Purpose
Locations	50	50	Campus graph vertices with name, area, type and local coordinates
Roads / edges	100	100	Weighted graph edges with distance, travel time and road-condition weight
Service requests	300	300	Queued/prioritized incidents with source, destination, category,

			urgency, deadline and status
Resources	30	30	Assignable officers/assets with type, home location, capacity and availability
Algorithm runs	30	30 planned seed records + full efficiency CSV evidence	Empirical runtime/memory metadata
Scenarios	Not separately mandated	12	Alternative road/routing conditions
Audit events	Recommended	60 seed/audit evidence	Workflow history and undo/audit support

2.2 Provenance and Privacy

The dataset uses recognizable University of Ghana location names and operational categories while avoiding personal or live incident data. Coordinates are documented as schematic/local coordinates. The repository evidence includes a data dictionary, dataset provenance note, validation report and manifest. This satisfies the project requirement for a Ghanaian local context and a short evidence note explaining how the data was constructed without exposing personal data.

2.3 Index-Number-Derived Parameters

The project documents all 15 team index numbers and derives three reproducible algorithm parameters from them. These parameters are not arbitrary constants; they influence operational and experimental behavior.

Table 3. Team-index-derived project parameters and their use

Parameter	Derivation	Value	Operational use
Priority weight	1 + (sum of final two digits mod 5); sum = 897	3	PriorityService / dispatch priority
Optimization budget	50 + (897 mod 51)	80	Dynamic-programming and brute-force optimization
Benchmark seed	Sum of final three digits of the 15 index numbers	7497	Deterministic efficiency-lab workload generation

2.4 Database Schema

The SQLite layer includes tables for locations, roads, service requests, resources, algorithm runs and audit events, with additional project-specific support for assignments and road scenarios. CSV import is transactional and validates required counts, identifiers, foreign keys and field constraints before the application relies on the data.

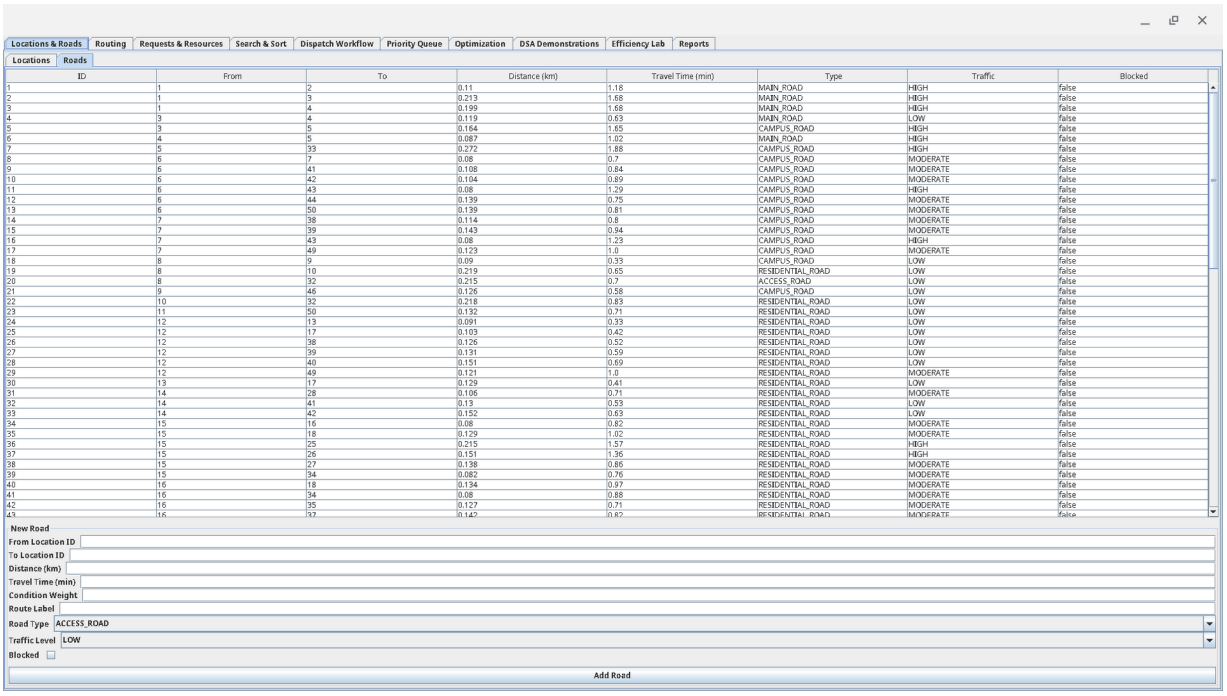

Figure 1. Weighted campus-road records used to build the emergency-response graph; the final UI exposes road/network maintenance under Operational Mode -> Routes -> Campus Network.

3. System Architecture and Module Design

The system follows a layered architecture so that operator-facing workflow, examiner-facing DSA evidence, business services, custom data structures and algorithms, persistence, and performance evidence remain separate. User or examiner actions begin in one of the two Swing presentation modes, pass through the same frontend service adapters, and are coordinated by shared application services. The services invoke the custom DSA/algorithm layer for assessed computation and use the persistence layer for runtime state, while correctness and efficiency components produce trace and benchmark evidence from the same implementations.

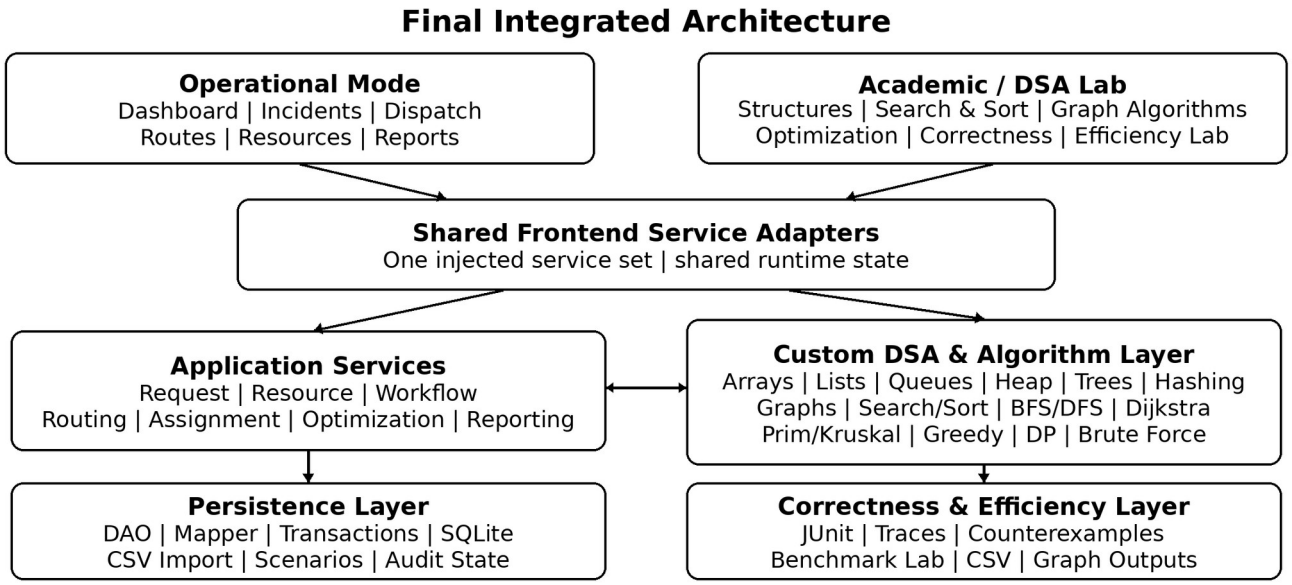

Figure 2. Layered architecture showing Operational Mode and Academic / DSA Lab sharing the same service, algorithm and persistence layers.

3.1 Architectural Layers and Responsibilities

Presentation layer. The Swing desktop interface is split into two views over the same runtime state. Operational Mode is the default and provides Dashboard, Incidents, Dispatch, Routes, Resources and Reports using user-facing terminology. Academic / DSA Lab exposes Structures, Search & Sort, Graph Algorithms, Optimization, Correctness and Efficiency Lab so examiners can inspect custom data structures, algorithms, traces and performance evidence without cluttering the normal operator workflow.

Frontend service adapters. Adapters form a stable boundary between Swing screens and backend services. They translate UI actions into service calls and return display-ready results without exposing database or structure internals to the interface.

Application service layer. Application services coordinate request priority, routing, resource assignment, workflow transitions, optimization, undo/audit operations, and reporting. This layer owns workflow decisions rather than the GUI.

Custom DSA and algorithm layer. Assessed structures and algorithms are implemented in project code: arrays, linked structures, queues, heap, trees, hashing, graphs, searching, sorting, graph algorithms, greedy assignment, dynamic programming, and brute force.

Persistence layer. DAO, mapper, transaction, and graph-loading components isolate SQLite/JDBC access from the algorithms. CSV seed data is validated and imported, while runtime changes are persisted for later reload and reporting.

Correctness and efficiency layer. Trace generation, correctness evidence, tests, and the efficiency lab observe the same implementations used by the application. Results are exported as CSV files, graphs, traces, and report evidence.

Design rationale. This separation keeps the assessed DSA logic reusable and testable, prevents UI code from becoming the source of business rules, and lets the same algorithms be exercised through workflow screens, correctness traces, and performance experiments.

Two-mode interface design. Operational Mode opens by default. A DSA Lab control switches the main content to the academic view, and Back to Operations returns to the operator view. The switch reuses the same injected service objects and does not create a second backend, duplicate database connection or separate application state.

3.2 Dispatch and Optimization Workflow

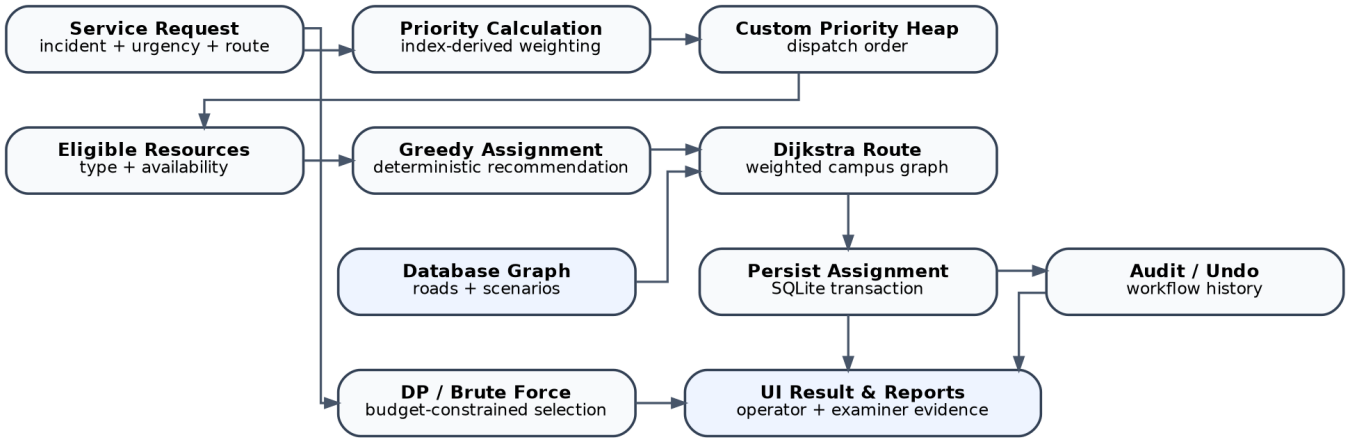

Figure 3. Emergency-dispatch workflow linking prioritisation, routing, persistence and optimisation.

A request can be prioritized, passed through custom priority scheduling, matched against eligible resources, routed through the database-backed graph, persisted as an assignment and recorded in the audit history. Separately, the optimization service can compare dynamic programming with brute force under the index-derived budget constraint.

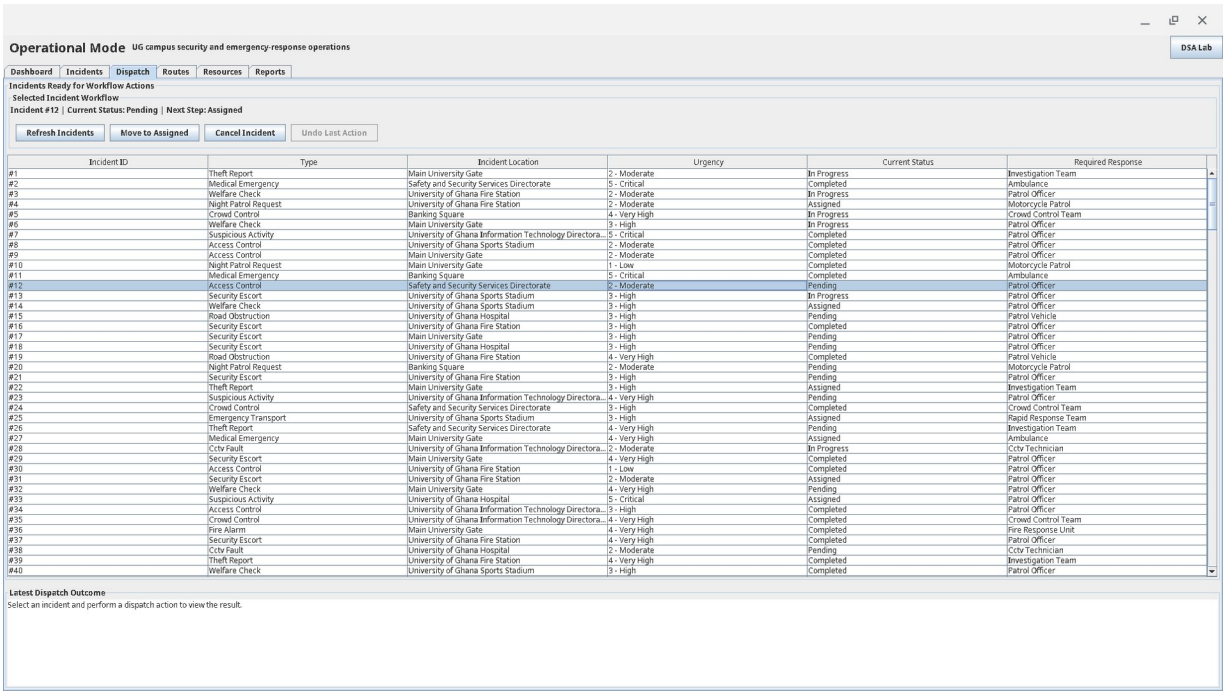

Figure 4. Operational Mode - Dispatch showing a selected pending incident, its current status, next workflow step and available action.

4. Data-Structure Implementation

All assessed core data structures were implemented in project code rather than replaced by prohibited Java collection classes. Standard library collections are used only where permitted, such as presentation, JDBC, testing or export scaffolding. Each required structure is supported by tests and project-specific evidence.

Table 4. Custom data structures: required behaviour and project evidence

Structure	Required behavior demonstrated	Project role / evidence
Dynamic array	insert, get, set, remove, resize	Dedicated implementation/tests and resize trace
Linked list + iterator	addFirst/addLast/insert/remove/ iteration	Iterator demo and linked-list evidence
Stack	push/pop/peek/isEmpty	Undo/audit demonstration
FIFO queue	enqueue/dequeue	FIFO request scheduling
Circular queue	wrap-around	Front/rear movement and rotation trace
Deque	front/rear add/remove	Urgent-front and routine-rear scheduling demonstration
Binary heap / priority queue	insert/extract/heapify	Urgent dispatch; also supports graph algorithms
BST	insert/search/inorder	Request indexing, search path and inorder output
Red-Black Tree	balanced insertion, rotations/recolouring	Request indexing, balancing/height evidence
B-Tree	split/search	Request indexing, root/split/search evidence
Hash table	put/get/remove/collision handling	Collision/load-factor experiments; custom map/set foundation
Custom map/set	membership/lookup	Dedicated tests and evidence
Disjoint Set	make/find/union, rank, path compression	Kruskal connectivity / MST support
Adjacency-list graph	weighted graph operations	Database graph loading; BFS/DFS/Dijkstra/MST

Adjacency-matrix graph	weighted graph operations	Alternative graph representation and reload tests

4.1 Scheduling Engine

The Academic / DSA Lab -> Structures view compares scheduling behavior using the actual custom FIFO queue, circular queue, deque and binary heap. This addresses the brief requirement that these structures model different dispatch rules rather than existing only as isolated classes.

4.2 Indexing Engine

The same Structures area exposes request indexes using the BST, Red-Black Tree and B-Tree. It shows search paths, inorder output, balancing/rotation information, height, and B-Tree split/search behavior. SQLite remains the system of record; the tree demonstrations illustrate in-memory indexing concepts rather than replacing the database.

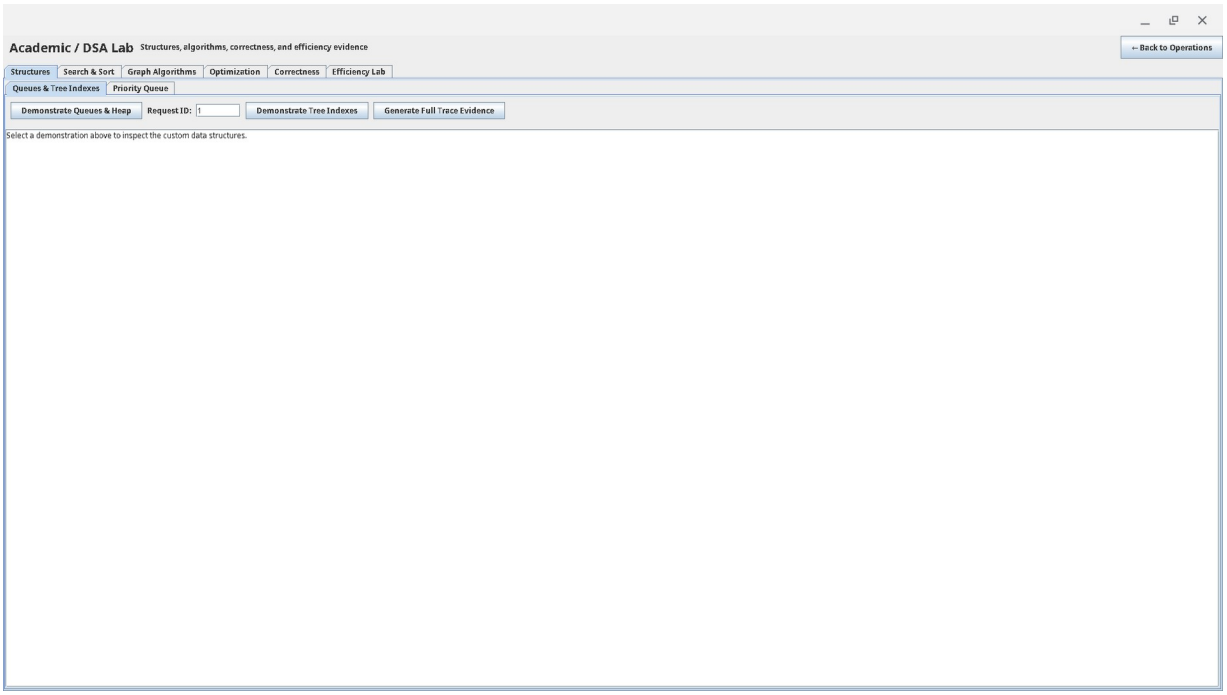

Figure 5. Academic / DSA Lab - Structures view exposing custom queue/heap and tree-index demonstrations.

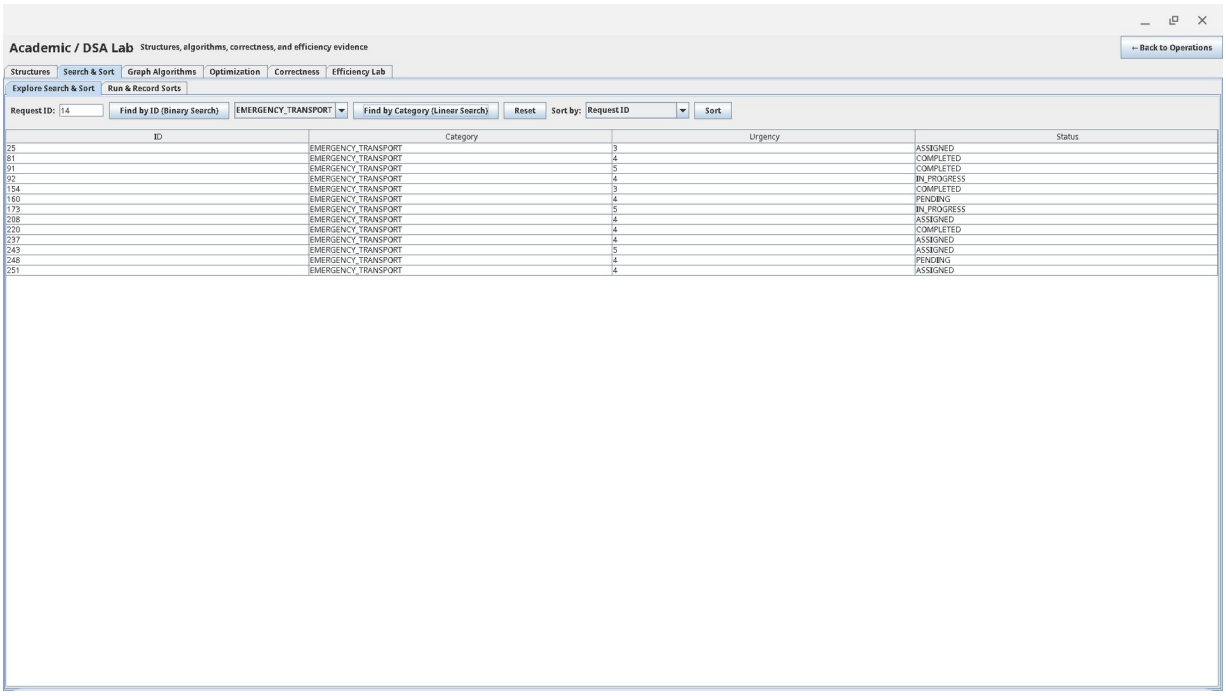

Figure 6. Academic / DSA Lab - Search & Sort view exposing Binary Search, Linear Search and sorting over request records.

5. Algorithm Implementation

The algorithm layer implements the required searching, sorting, graph, greedy and dynamic-programming strategies, with brute force included as an exhaustive exact baseline for small optimization instances.

Table 5. Implemented algorithms: project use and theoretical complexity

Algorithm	**Project use**	**Standard theoretical complexity (reference)**
Linear Search	Search demonstration / benchmark	O(n) time
Binary Search	Sorted-data search / benchmark	O(log n) time after sorting/precondition
Selection Sort	Sorting comparison	O(n^2) time, in-place
Insertion Sort	Sorting comparison	O(n^2) worst-case; adaptive on nearly sorted input
Merge Sort	Sorting comparison	O(n log n) time; additional merge storage
Quick Sort	Sorting comparison and worst-case demo	Average O(n log n), worst O(n^2)
BFS	Reachability/traversal	O(V+E) with adjacency list
DFS	Reachability/traversal	O(V+E) with adjacency list
Dijkstra	Weighted shortest path	Common heap-based form O((V+E) log V)
Prim	Minimum spanning network	Depends on representation/priority structure; typically near O(E log V)
Kruskal	Minimum spanning network	Typically O(E log E) dominated by edge sorting
Greedy assignment	Resource recommendation	Project-specific candidate evaluation; fast but not globally optimal for every objective
Dynamic Programming	Budget/capacity request selection	Pseudo-polynomial in item count x capacity

Brute Force	Exact comparison baseline	Exponential in item count

The complexity values in the third column are standard theoretical algorithm properties and are intentionally kept separate from the measured Java runtimes reported in Section 7.

5.1 Search and Sort

Binary Search explicitly requires sorted input; the project includes an invalid-precondition counterexample using unsorted input. Selection and Insertion Sort provide simple in-place quadratic baselines, while Merge Sort and Quick Sort provide divide-and-conquer comparisons.

5.2 Graph Algorithms

BFS and DFS expose reachability and deterministic traversal order. Dijkstra is integrated into the routing service and Swing routing screen. Prim and Kruskal are retained as network-analysis algorithms and are demonstrated through correctness evidence and the efficiency lab rather than being forced into the dispatch workflow.

5.3 Greedy, DP and Brute Force

Greedy assignment supports actual resource recommendation, while dynamic programming solves a capacity/budget selection problem using the index-derived budget 80. Brute force solves the same exact optimization objective for safe small inputs, providing both a correctness baseline and a tractability comparison. The project includes a deterministic counterexample showing that a greedy local choice can be globally suboptimal.

5.4 Selected Pseudocode

The following project-adapted pseudocode defines the inputs, outputs and decision steps for six major operations. It complements the trace evidence in Section 6 and makes the system behavior explicit without relying on source-code inspection.

Binary Search over sorted request IDs

```
INPUT: sorted request array A, target request ID x
OUTPUT: index of x, or -1 if absent
low <- 0; high <- length(A) - 1
WHILE low <= high
    mid <- low + (high - low) / 2
    IF A[mid] = x: RETURN mid
    IF A[mid] < x: low <- mid + 1
    ELSE: high <- mid - 1
RETURN -1
```

Priority dispatch using the custom binary heap

```
INPUT: pending service requests
OUTPUT: requests removed in dispatch-priority order
heap <- empty custom BinaryHeap
FOR each pending request r
    compute project priority using urgency and tie-breaking rules
    INSERT r into heap
WHILE heap is not empty
    r <- EXTRACT highest-priority request
    present or dispatch r
END
```

Dijkstra scenario-aware routing

```
INPUT: weighted campus graph G, source s, destination t
OUTPUT: minimum modeled-cost path, or unreachable
distance[*] <- infinity; predecessor[*] <- none
distance[s] <- 0; INSERT (0,s) into custom heap
WHILE heap is not empty
    u <- vertex with smallest current distance
    IF u is stale/settled: CONTINUE
    settle u; IF u = t: BREAK
    FOR each incident edge (u,v,w)
        candidate <- distance[u] + w
        IF candidate < distance[v]
            distance[v] <- candidate; predecessor[v] <- u
            INSERT updated (distance[v],v) into heap
RECONSTRUCT path from predecessor[]
```

Kruskal minimum spanning forest

```
INPUT: weighted campus graph G
OUTPUT: minimum spanning forest edge set and total cost
edges <- independent snapshot of graph edges
SORT edges by weight with deterministic tie-breaking
make a DisjointSet containing every vertex
forest <- empty; total <- 0
FOR each edge (u,v,w) in sorted order
    IF FIND(u) != FIND(v)
        add edge to forest; UNION(u,v); total <- total + w
RETURN forest, total, component count
```

Dynamic-programming request selection

```
INPUT: candidate requests/items, capacity C
OUTPUT: exact subset with maximum urgency benefit
create DP benefit/cost tables for states (itemIndex, availableCapacity)
FOR itemIndex from n-1 down to 0
    FOR available from 0 to C
        excluded <- value of skipping item
        IF item cost <= available
            included <- item benefit + value of remaining capacity
            choose better of included/excluded using deterministic tie rules
        ELSE choose excluded
RECONSTRUCT selected items from stored decisions
RETURN optimal subset
```

Greedy resource assignment

```
INPUT: pending requests, available compatible resources, weighted graph
OUTPUT: deterministic request-resource recommendations
ORDER requests by dispatch priority
FOR each request r
    evaluate currently available resources compatible with r
    estimate modeled route/response cost for each candidate
    choose the locally best feasible resource using deterministic tie-breaking
    record assignment and mark resource unavailable for this pass
RETURN assignments
NOTE: compare against the stored counterexample; greedy is not guaranteed globally optimal.
```

5.5 Primitive Operation Counts and Asymptotic Summary

Two representative search algorithms were counted using key comparisons as the primitive operation. For Linear Search over n records, the best case needs 1 key comparison, an unsuccessful search needs n, and a successful search with the target equally likely in any position needs (n + 1) / 2 comparisons on average. Therefore its best-case lower bound is Omega(1), while average and worst behavior are Theta(n), with O(n) as the upper bound.

For Binary Search, each key comparison removes approximately half of the remaining sorted search interval. After k comparisons the interval is at most n / 2^k, so the worst-case number of iterations is at most floor(log2(n)) + 1 for n >= 1. Its best case is 1 comparison, giving Omega(1),

while average and worst behavior are Theta(log n), with O(log n) as the upper bound. This count assumes the required sorted-input precondition already holds.

Algorithm / operation	O upper bound	Theta (tight stated case)	Omega lower bound	Interpretation
Linear Search	O(n)	Theta(n) average/worst	Omega(1)	Sequential scan; unsuccessful search checks n keys.
Binary Search	O(log n)	Theta(log n) average/worst	Omega(1)	Requires sorted input; halves interval each step.
Selection Sort	O(n^2)	Theta(n^2)	Omega(n^2)	Always performs quadratic key comparisons.
Insertion Sort	O(n^2)	Theta(n^2) average/worst	Omega(n)	Best case is already sorted input.
Merge Sort	O(n log n)	Theta(n log n)	Omega(n log n)	Divide-and-conquer with linear merging per level.
Quick Sort	O(n^2) worst	Theta(n log n) average	Omega(n log n)	Partition quality determines recursion balance.
Heap insert/extract	O(log n)	Theta(log n) worst	Omega(1)	Heap height is logarithmic; no movement may be needed in best case.
BFS / DFS	O(V + E)	Theta(V + E) for full reachable traversal	Omega(1)	Visits reachable vertices/edges once under standard adjacency representation.
DP request selection	O(nC)	Theta(nC)	Omega(nC)	Bottom-up implementation evaluates every item-capacity state.
Brute-force selection	O(2^n)	Theta(2^n)	Omega(2^n)	Enumerates all candidate subsets in the exhaustive baseline.
Kruskal MST	O(E log E)	Theta(E log E) with comparison edge sort	Omega(E)	Sorting dominates typical implementation; DSU operations are near constant amortized.

Notation note: O is used as an asymptotic upper bound, Omega as a lower bound, and Theta where a tight bound is stated for the identified case. The measured results in Section 7 are empirical Java runtimes and are interpreted against, rather than substituted for, these theoretical growth rates.

5.6 Selected Java Implementation Snippets

The following excerpts are taken from the project source on the audited branch. They are intentionally short; the complete implementations and tests remain in the repository.

Binary Search - src/main/java/org/ugoptimizer/algorithms/BinarySearch.java

```
public static <T extends Comparable<T>> int search(T[] array, T target) {
    if (array == null || target == null) return -1;
    int low = 0;
    int high = array.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        T element = array[mid];
        if (element == null) return -1;
        int comparison = element.compareTo(target);
        if (comparison == 0) return mid;
        if (comparison < 0) low = mid + 1;
        else high = mid - 1;
    }
```

```
    return -1;
}
```

Dijkstra relaxation loop - src/main/java/org/ugoptimizer/algorithms/shortestpath/Dijkstra.java

```
while (!heap.isEmpty()) {
    DistanceEntry entry = heap.poll();
    int currentIndex = entry.vertexIndex;
    if (settled[currentIndex]
            || Double.compare(entry.distance, distances[currentIndex]) != 0) {
        continue;
    }
    settled[currentIndex] = true;
    if (currentIndex == destinationIndex) break;

    Edge[] incidentEdges = graph.getIncidentEdges(entry.vertexId);
    for (Edge edge : incidentEdges) {
        int neighborId = oppositeVertexId(edge, entry.vertexId);
        int neighborIndex = indexOf(vertexIds, neighborId);
        if (settled[neighborIndex]) continue;
        double candidateDistance = entry.distance + edge.getWeight();
        if (Double.compare(candidateDistance, distances[neighborIndex]) < 0) {
            distances[neighborIndex] = candidateDistance;
            predecessorIndexes[neighborIndex] = currentIndex;
            heap.add(new DistanceEntry(neighborIndex, neighborId, candidateDistance));
        }
    }
}
```

Dynamic Programming state transition - src/main/java/org/ugoptimizer/algorithms/optimization/DynamicProgrammingIncidentSelector.java

```
for (int itemIndex = items.length - 1; itemIndex >= 0; itemIndex--) {
    OptimizationItem item = items[itemIndex];
    for (int available = 0; available <= capacity; available++) {
        long excludedBenefit = bestBenefits[itemIndex + 1][available];
        long excludedCost = bestCosts[itemIndex + 1][available];
        bestBenefits[itemIndex][available] = excludedBenefit;
        bestCosts[itemIndex][available] = excludedCost;

        if (item.getCost() <= available) {
            int remaining = available - item.getCost();
            long includedBenefit = checkedAdd(
                    bestBenefits[itemIndex + 1][remaining], item.getBenefit(), "benefit");
            long includedCost = checkedAdd(
                    bestCosts[itemIndex + 1][remaining], item.getCost(), "cost");
            if (isBetter(includedBenefit, includedCost,
                         excludedBenefit, excludedCost)) {
                bestBenefits[itemIndex][available] = includedBenefit;
                bestCosts[itemIndex][available] = includedCost;
                selectedAtState[itemIndex][available] = true;
            }
        }
    }
}
```

These excerpts show three assessed patterns directly in project code: halving a sorted search interval, relaxing weighted graph edges with the custom heap, and filling/reconstructing a bottom-up 0/1 knapsack-style DP table. The corresponding trace tables and tests are presented or referenced in Section 6.

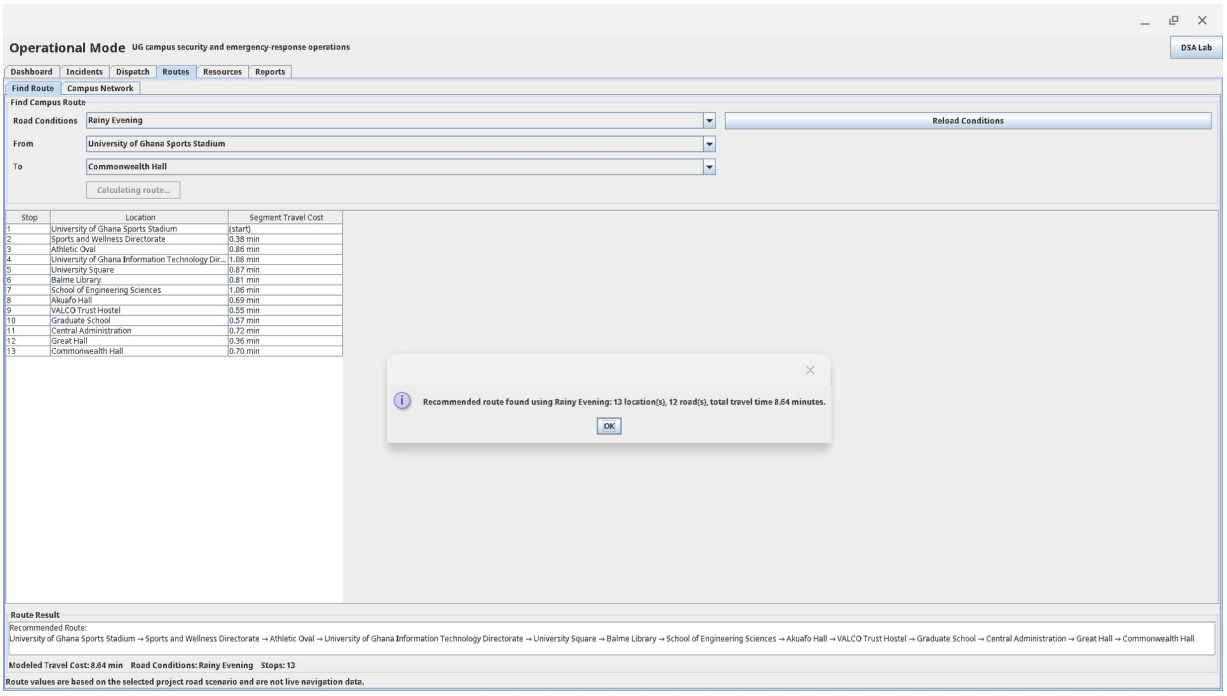

Figure 7. Operational Mode - Routes showing the Dijkstra shortest path from the UG Sports Stadium to Commonwealth Hall under Rainy Evening; modeled travel cost: 8.64 minutes.

The 8.64-minute value is an algorithmic result within the project weighted graph. It is not presented as a live navigation or real-world travel-time guarantee.

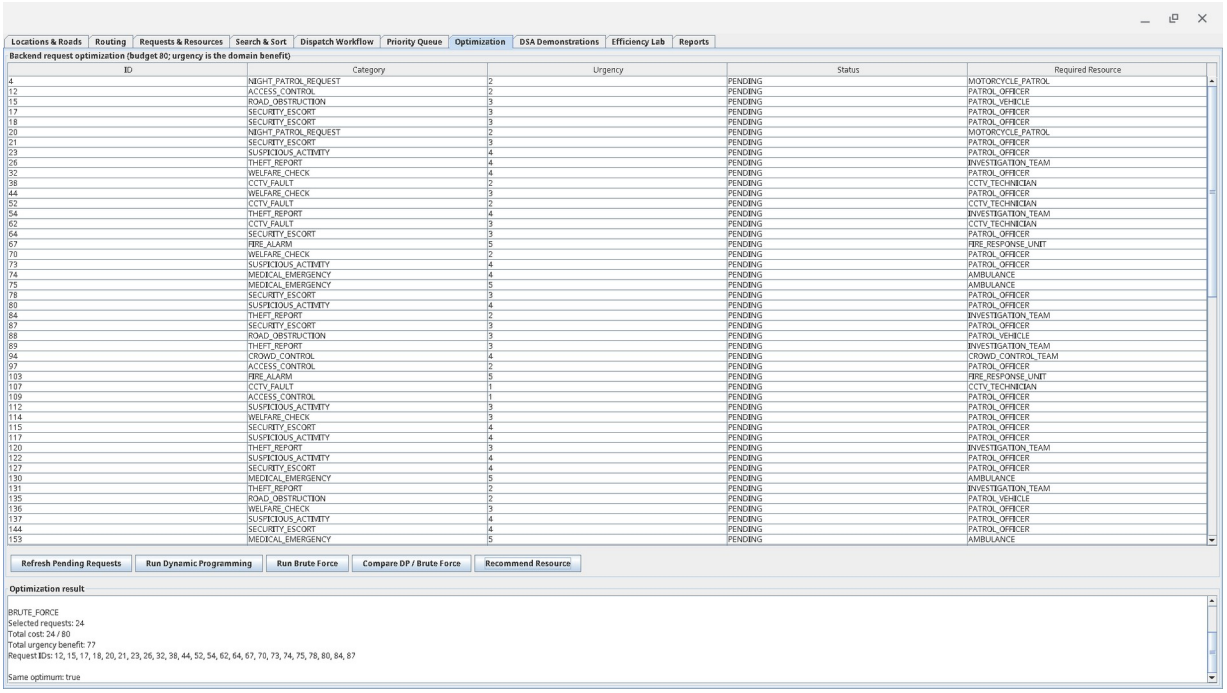

Figure 8. Academic / DSA Lab - Optimization comparing Dynamic Programming and Brute Force under budget 80; both exact methods return the same optimum.

6. Correctness Evidence

Correctness evidence combines unit tests, deterministic trace output, proof sketches, explicit preconditions and counterexamples. The final verified suite contains 681 passing tests with no failures, errors or skipped tests.

Table 6. Correctness evidence summary

Evidence type	Project evidence
Unit tests	681 passing tests covering data structures, algorithms, database, services, frontend adapters and end-to-end workflow
Trace tables	Binary Search, Insertion Sort, Merge/Quick Sort, Dijkstra, Prim/Kruskal and Dynamic Programming, plus structure-specific traces
Proof sketches	Binary-search invariant, merge-sort induction/recursion argument, greedy/DP correctness ideas and graph notes
Counterexamples	Greedy failure and unsorted-input Binary Search precondition violation
Edge cases	Empty/single-element structures, duplicate keys, disconnected graph, unreachable path, queue full/empty and hash collisions

6.1 Required Trace Set

1. Binary Search trace showing low/high/mid progression.
2. Insertion Sort trace showing the growing sorted prefix.
3. Merge Sort or Quick Sort trace showing decomposition and reconstruction/partition behavior.
4. Dijkstra trace showing distance/predecessor updates and reconstructed path.
5. Prim or Kruskal trace showing accepted MST edges and total cost.
6. Dynamic Programming trace showing tabulation/reconstruction under the budget constraint.

6.2 Proof Sketches

Binary Search invariant: before each iteration, if the target exists it remains within the current search interval. Comparing with the middle value discards only a half that cannot contain the target, preserving the invariant until the target is found or the interval becomes empty.

Merge Sort induction idea: a one-element list is already sorted. Assuming recursive calls correctly sort smaller halves, merging two sorted halves by repeatedly taking the smaller front element produces a sorted combined list containing exactly the original elements.

Dynamic Programming correctness idea: the table stores the best achievable objective value for each prefix of items and capacity. For each state the recurrence considers both excluding and, when feasible, including the current item. Because these are the only two possibilities, the maximum gives an optimal subproblem result; reconstruction follows the choices that created the optimal final state.

6.3 Counterexamples and Limits

Greedy algorithms can make a locally attractive assignment or selection that blocks a better overall combination. The project therefore retains a deterministic greedy failure example and uses DP/brute force when the task requires an exact budget-constrained optimum. Binary Search provides the second required counterexample: on unsorted input its ordering precondition is violated, so the half-discarding logic is not valid.

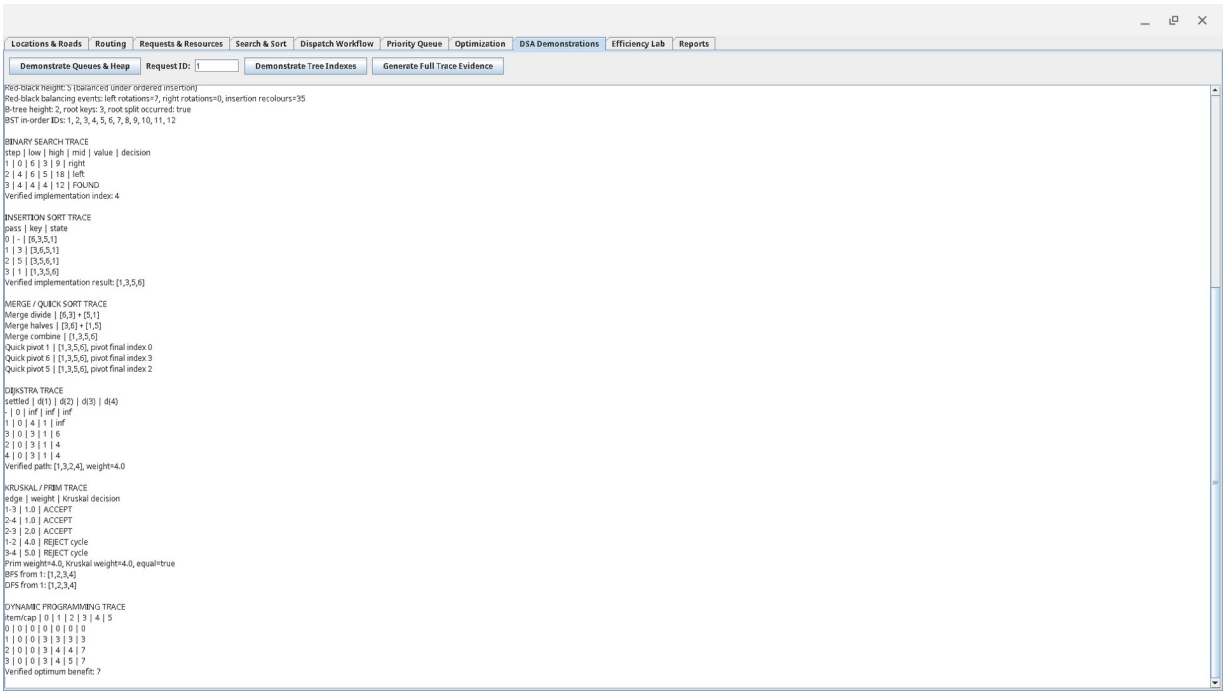

Figure 9. Generated correctness traces for search, sorting, shortest path, MST, traversal and dynamic programming.

7. Performance Analysis

7.1 Experimental Setup and Methodology

The final official-size efficiency lab was executed on one machine and retained as raw CSV evidence. Workloads are deterministic through benchmark seed 7497. Each experiment group contains three measured trials and reports an average. Setup, correctness checks and diagnostic collection are kept outside the timed region where practical. Runtime uses high-resolution nanosecond timing. Memory values are approximate used-heap deltas; negative deltas are clamped to zero and are not interpreted as exact theoretical space complexity.

Table 7. Benchmark environment and execution settings

Environment item	Value
CPU	12th Gen Intel(R) Core(TM) i3-1215U
RAM	6.3 GiB
Swap	0 B
OS	Linux 6.6.135-09383-g1140e4f27e24
Architecture	amd64
Java	17.0.20
JVM	OpenJDK 64-Bit Server VM
Processors available to JVM	8
Maximum JVM heap	1,696,595,968 bytes (~1.58 GiB)
Benchmark seed	7497
Trials per benchmark group	3

The full run produced 336 genuine raw trial rows. The original raw evidence is retained in results/full-efficiency-lab/benchmark-results.csv; averaged report-ready data is retained in benchmark-summary.csv.

7.2 Search Performance

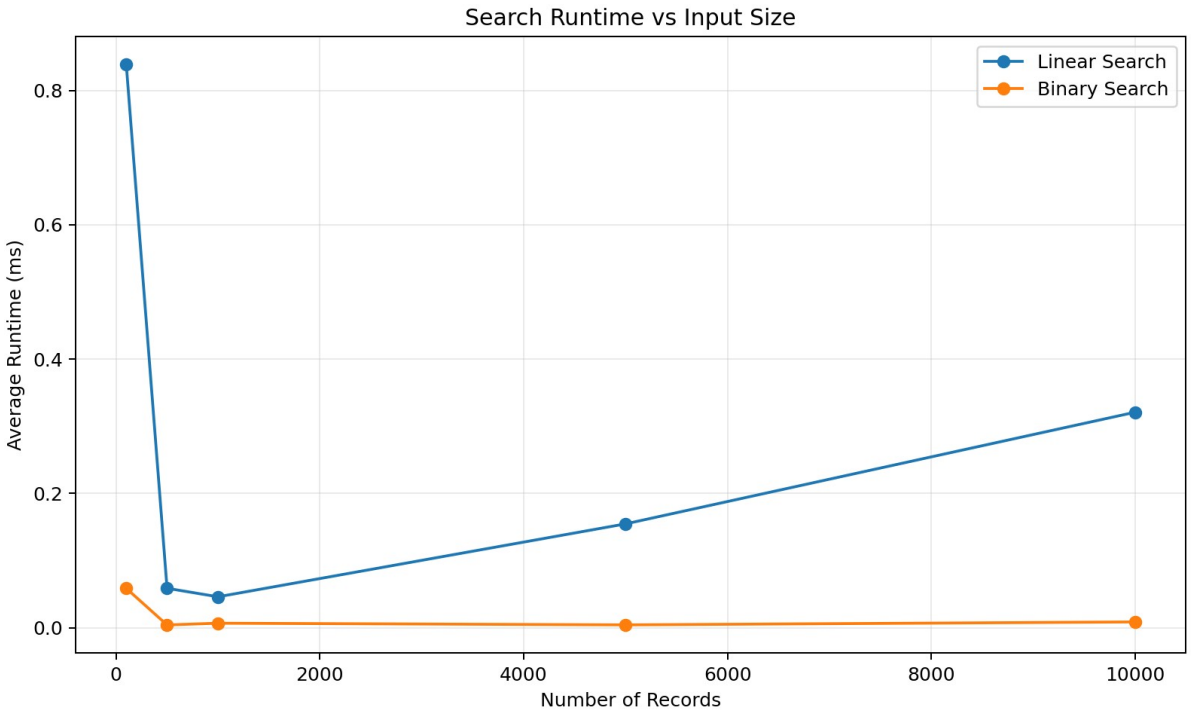

Figure 10. Linear Search and Binary Search runtime across increasing input sizes.

Binary Search outperformed Linear Search as the dataset grew. At 10,000 records, Linear Search averaged approximately 0.321 ms, while Binary Search averaged 0.0085 ms. This supports the expected O(n) versus O(log n) search-growth difference when Binary Search is given sorted input. The relatively high first measurements at small sizes illustrate JVM/JIT warm-up effects, so the growth pattern across sizes is more informative than a single small-n point.

7.3 Sorting Performance

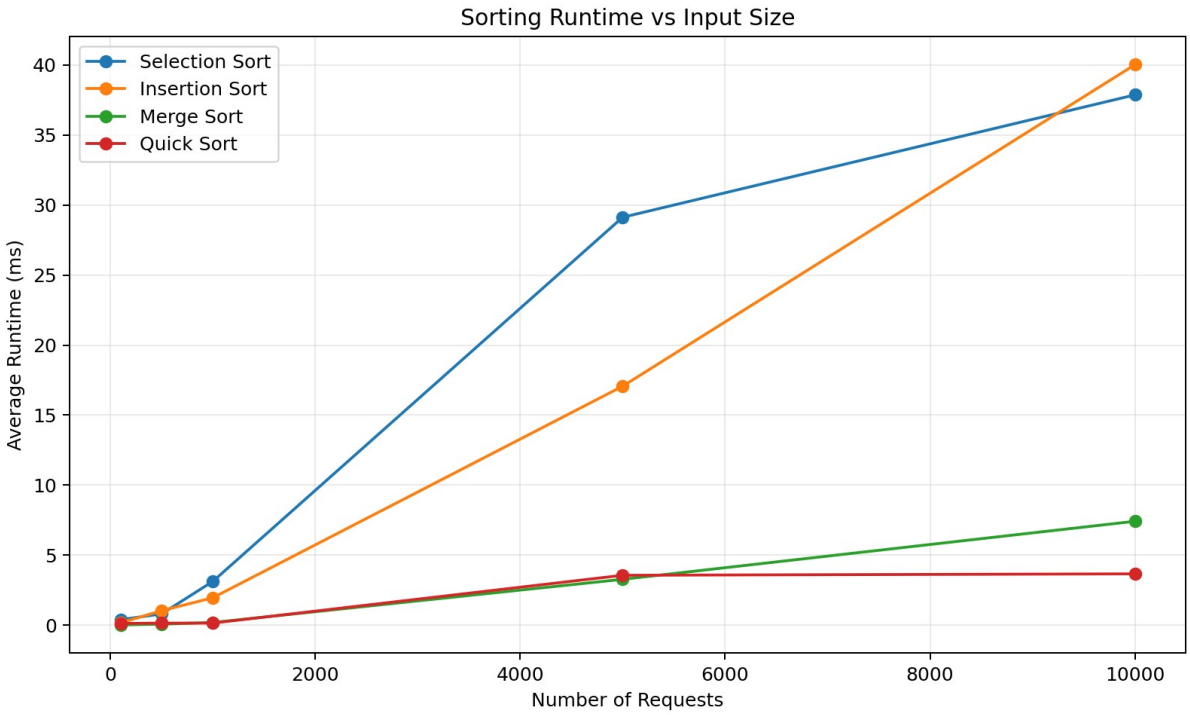

Figure 11. Runtime comparison of Selection, Insertion, Merge and Quick Sort.

At 10,000 elements, Selection Sort averaged 37.9 ms and Insertion Sort 40.0 ms. Merge Sort averaged 7.4 ms and Quick Sort 3.7 ms. The quadratic algorithms become much more expensive as n increases, while the divide-and-conquer algorithms scale more effectively on the deterministic random-order workloads. Small timing irregularities are expected from JVM execution and input-specific constant factors.

7.4 Hash Table Performance

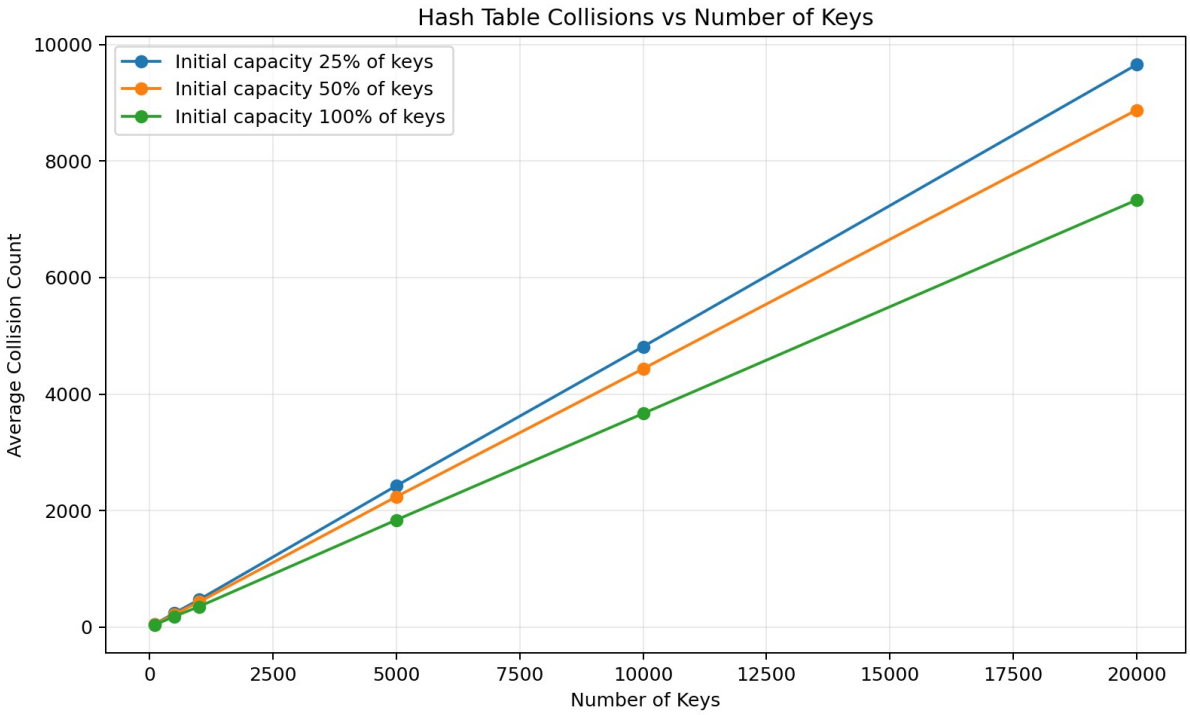

Figure 12. Hash-table collision behaviour under different initial capacities.

The hash-table experiment varies the number of keys and the initial table capacity. Higher effective load produces more collisions and can increase operation time. Collision count is the most stable evidence because short runtime measurements also reflect resizing, allocation, cache effects and JVM activity. One 10,000-key timing group shows a noticeable spike; it is retained rather than removed because the brief requires genuine empirical evidence and interpretation of mismatch/noise.

7.5 BST versus Red-Black Tree

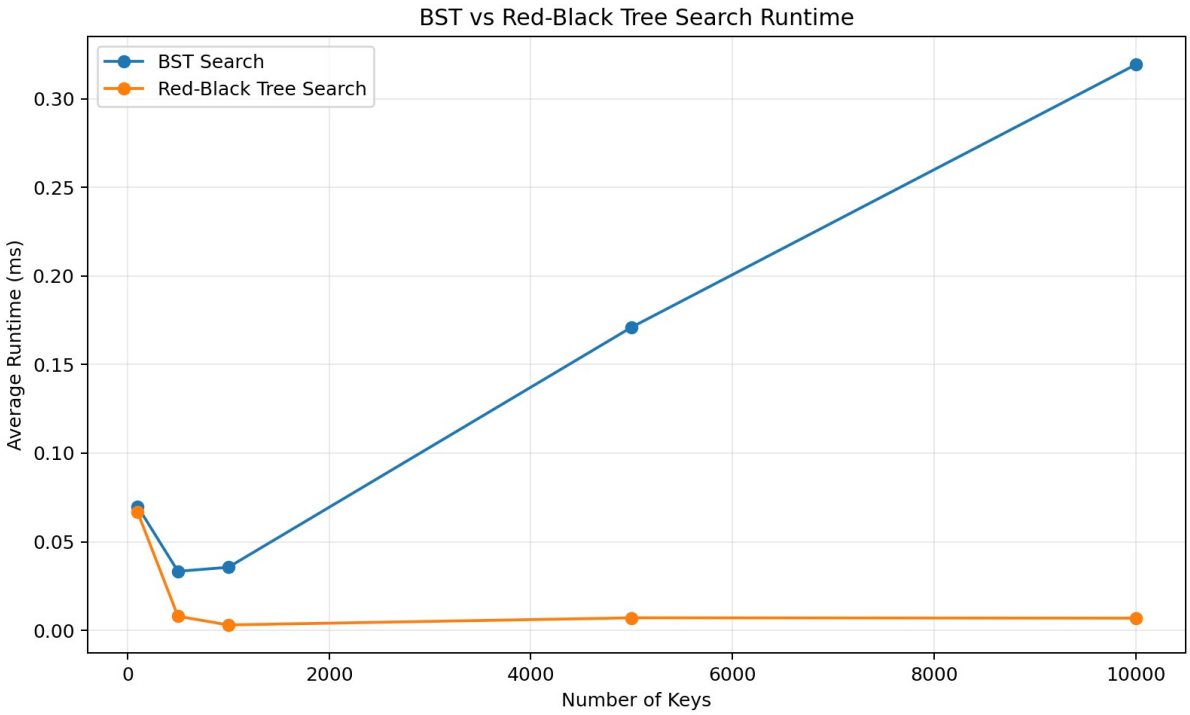

Figure 13. BST and Red-Black Tree search runtime under ordered-key workloads.

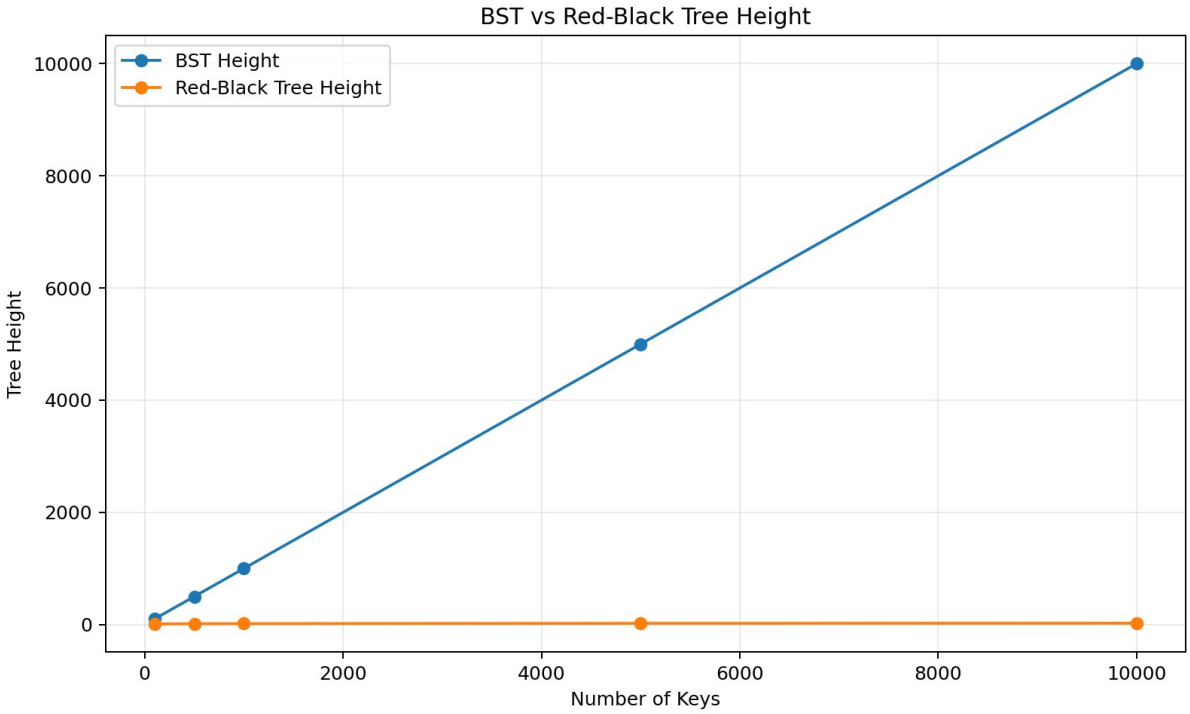

Figure 14. BST and Red-Black Tree height under ordered insertion.

This is one of the clearest demonstrations. Ordered insertion makes the ordinary BST degenerate: at 10,000 keys its height reaches 10,000, while the Red-Black Tree remains around 24. BST insertion at 10,000 keys averaged 67.4 ms compared with 0.90 ms for the Red-Black Tree. Search averaged

0.319 ms versus 0.0069 ms. The measured results illustrate why balancing protects operations from degrading toward linear-height behavior.

7.6 Heap Priority Dispatch

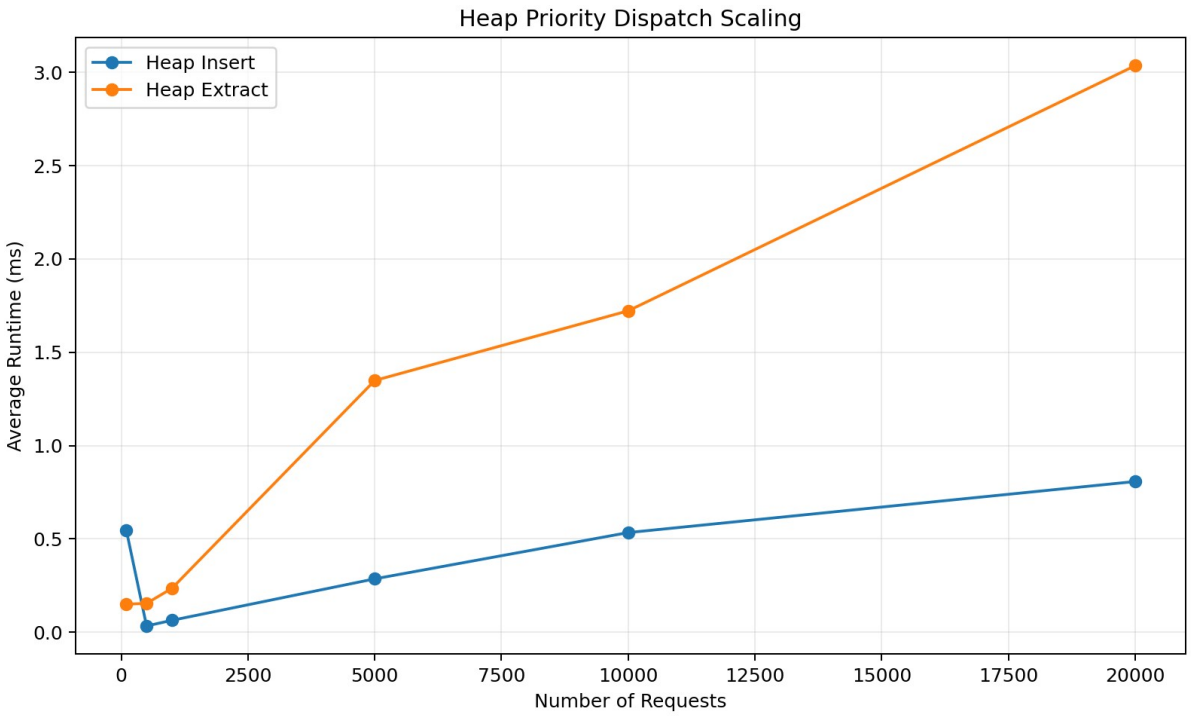

Figure 15. Scaling of custom binary-heap insertion and extraction.

Heap extraction rises with workload size and averages approximately 3.04 ms for the complete 20,000-request extraction benchmark group. Heap insertion also increases overall, reaching about 0.81 ms. The 100-request insertion point is relatively noisy, but the overall trend remains consistent with accumulating logarithmic heap operations over larger workloads. This supports the project choice of a heap for priority dispatch rather than repeatedly fully sorting the pending-request set.

7.7 Graph Algorithms

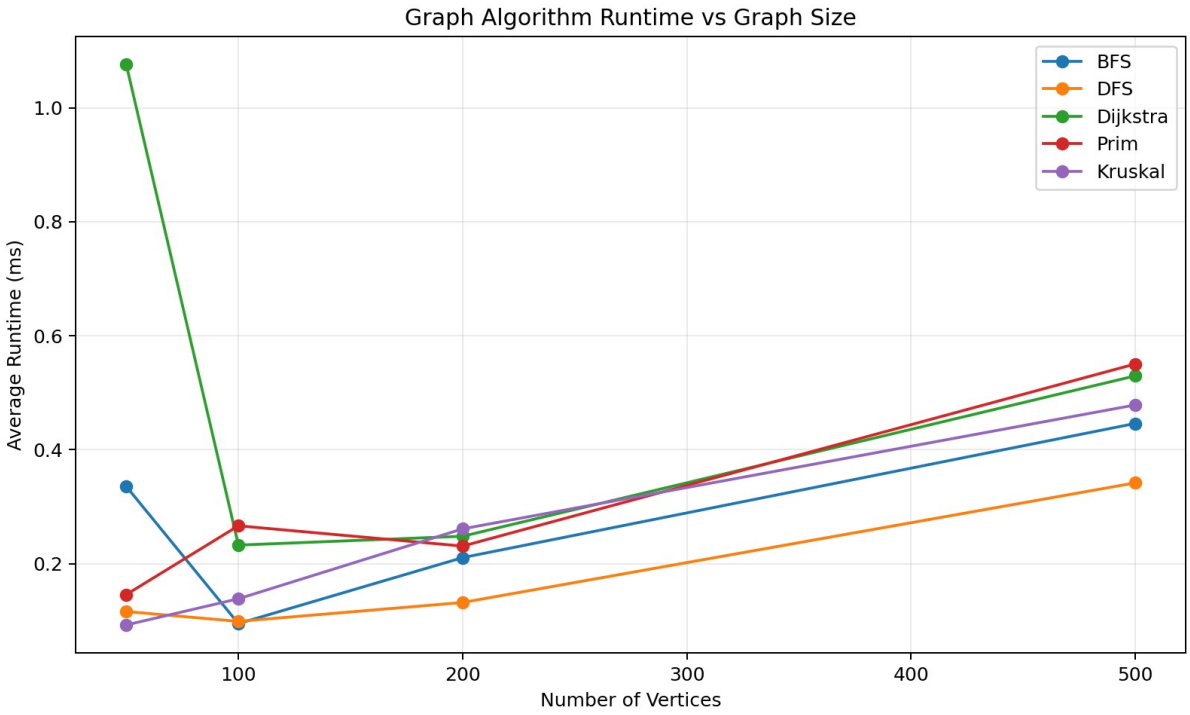

Figure 16. Runtime comparison of BFS, DFS, Dijkstra, Prim and Kruskal.

At 500 vertices, average times were approximately 0.446 ms for BFS, 0.342 ms for DFS, 0.530 ms for Dijkstra, 0.550 ms for Prim and 0.479 ms for Kruskal. BFS and DFS remain relatively inexpensive because they perform traversal, while Dijkstra and the MST algorithms maintain additional weighted-path or spanning-tree state. Prim and Kruskal produced matching MST total weights on compatible connected benchmark graphs, adding correctness evidence to the runtime comparison.

7.8 Dynamic Programming versus Brute Force

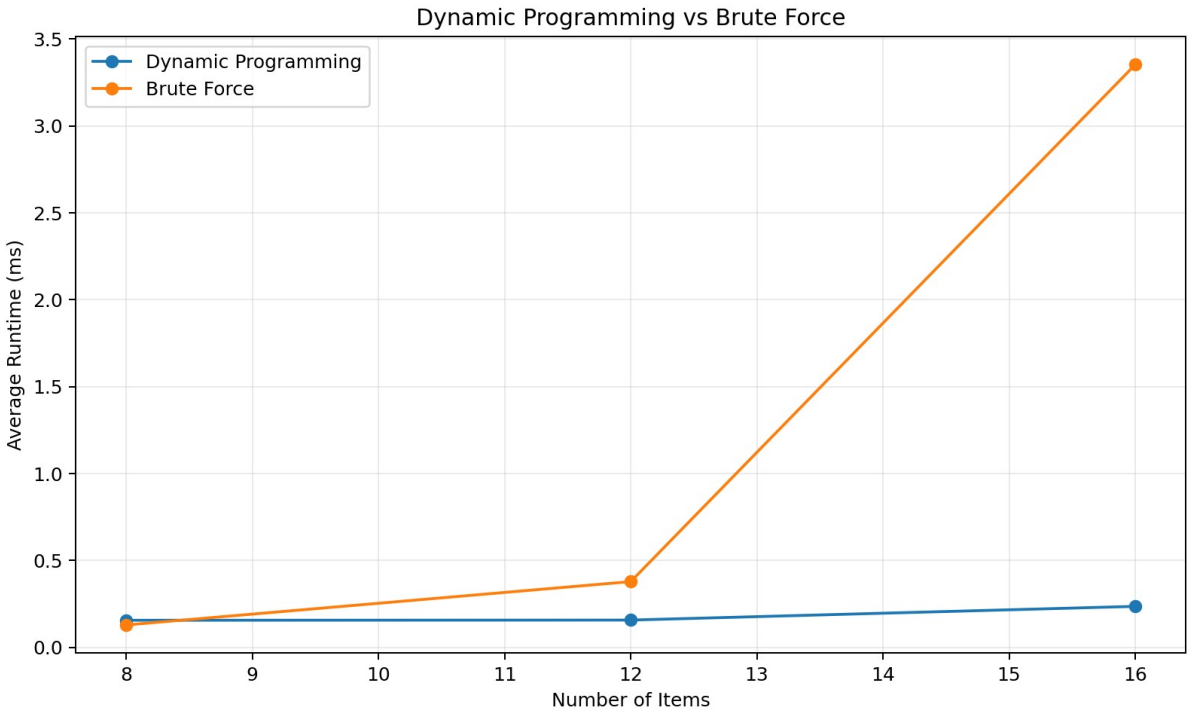

Figure 17. Dynamic Programming and Brute Force runtime as item count increases.

At 8 items the two exact methods are both small enough that overhead dominates. By 16 items, Brute Force averaged 3.35 ms while Dynamic Programming averaged 0.236 ms. Both methods agree on the optimal objective value, but the exhaustive method must examine a rapidly growing number of combinations. This makes the DP approach more suitable as problem size grows while retaining brute force as a small-n correctness baseline.

7.9 Greedy Assignment

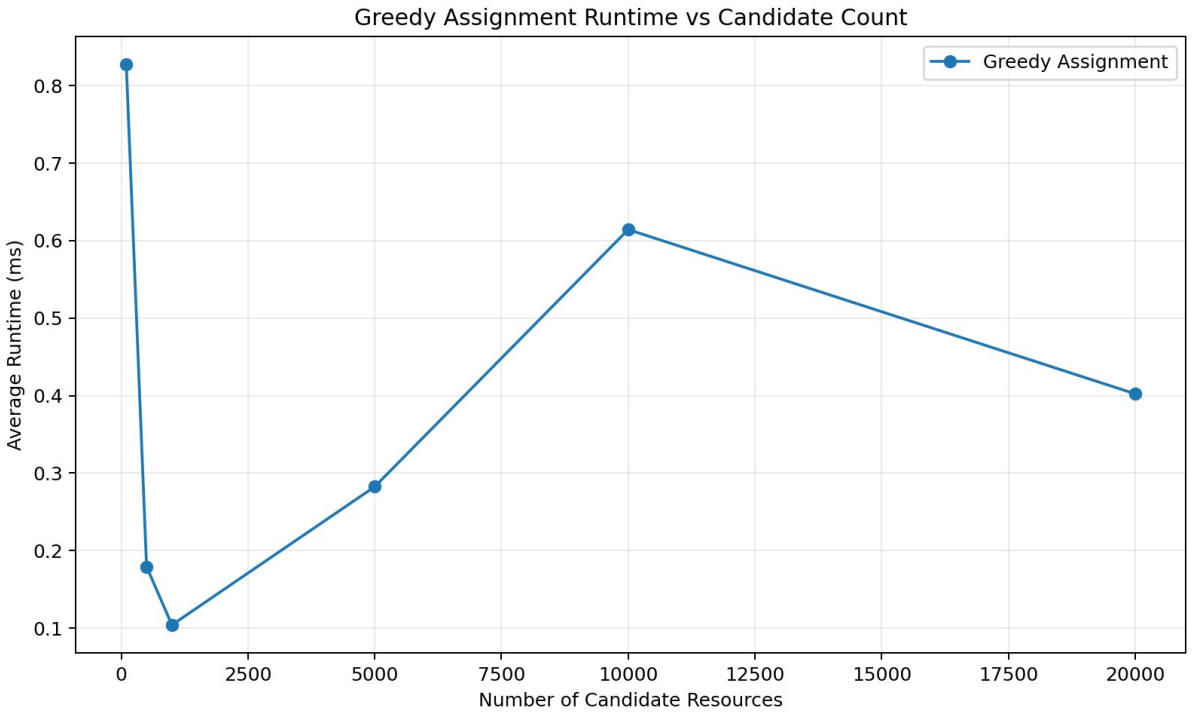

Figure 18. Greedy-assignment runtime as candidate-resource count increases.

Greedy assignment remains fast across the tested candidate counts. At 10,000 candidates it averaged 0.614 ms, while the 20,000-candidate result was 0.402 ms. The non-monotonic decrease at the largest size is retained as real evidence: short Java measurements are influenced by JIT compilation, cache state, garbage collection and operating-system scheduling. The important observation is that the implementation remains operationally small in runtime at the tested scales.

7.10 Overall Theory-versus-Empirical Interpretation

Overall, the experiments support the expected theoretical growth relationships but do not form perfectly smooth curves. This is normal for application-level Java benchmarking. JIT compilation, heap allocation, garbage collection, processor caching, background system activity and very short measured durations all affect individual timings. Three-trial averaging reduces some noise, while the fixed seed 7497 ensures equivalent workloads can be reproduced. Therefore, the analysis focuses on scaling trends, structural evidence such as tree height and collision counts, and agreement of correctness metrics rather than assuming every timing must rise monotonically.

Performance evidence location
Raw trials: results/full-efficiency-lab/benchmark-results.csv. Averaged summary: results/full-efficiency-lab/benchmark-summary.csv. Machine metadata: results/full-efficiency-lab/environment.txt. Final graphs: results/full-efficiency-lab/graphs/.

8. Database Integration Evidence

SQLite is part of the running system rather than a static submission artifact. The application initializes the schema, validates and transactionally imports the canonical CSV dataset, reads and writes operational records through DAOs, reloads graph structures from stored location/road data, persists assignments and audit/history events, and retains algorithm-run evidence or exports it to CSV as permitted by the brief.

Table 8. Runtime database integration evidence

Database responsibility	Evidence
Schema and initialization	Database manager initializes required tables and application state
CSV import	Transactional importer validates schema, counts, IDs, foreign keys and constraints
Persistent CRUD/workflow	Location, road, request, resource, assignment/audit operations backed by DAOs
Graph reload	DatabaseGraphLoader reconstructs adjacency-list/matrix graph data
Algorithm evidence	AlgorithmRun persistence and full CSV benchmark export; recorded runs are exposed in Academic / DSA Lab -> Efficiency Lab
Correctness	DAO, importer, database manager and end-to-end persistence tests included in passing suite

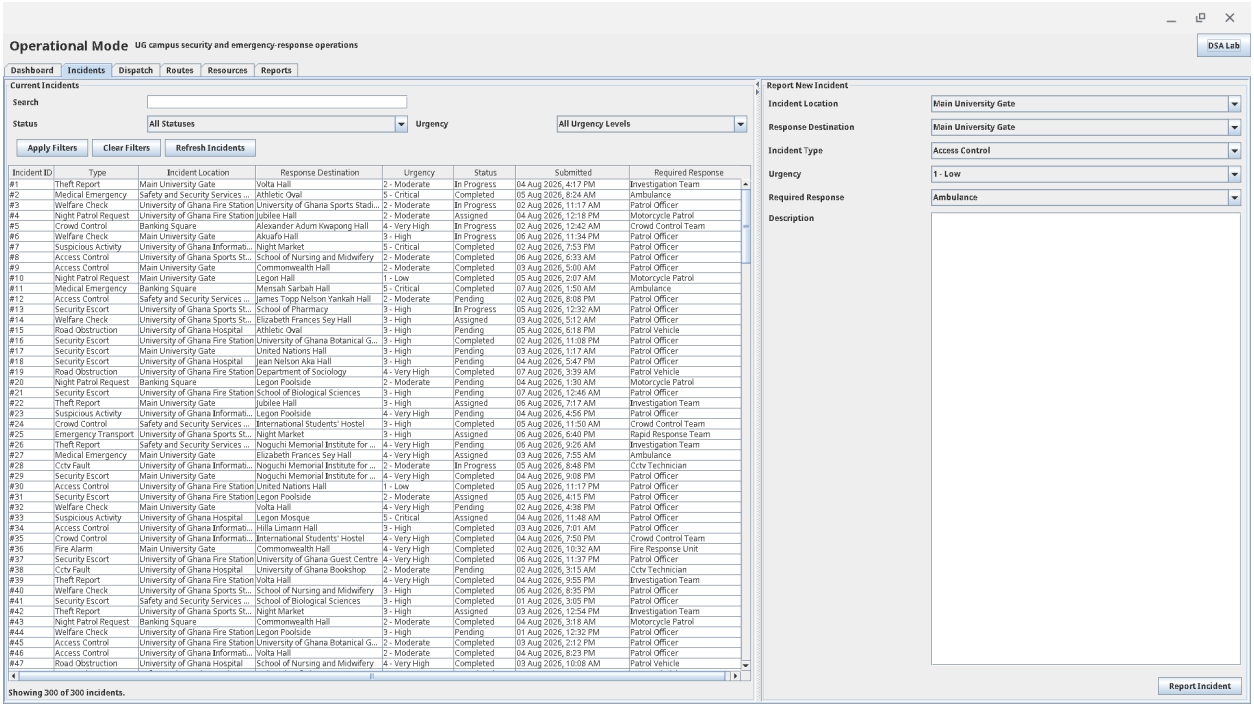

Figure 19. Operational Mode - Incidents showing SQLite-backed service requests with named campus locations, readable urgency/status values, filtering and incident reporting.

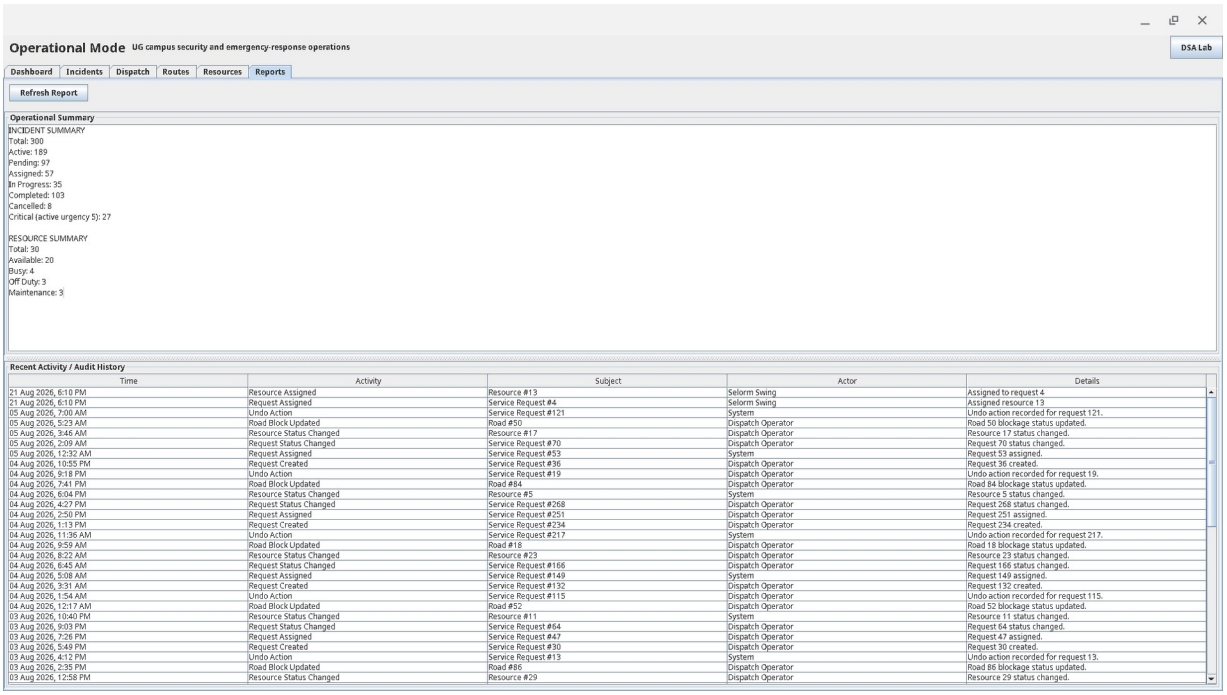

Figure 20. Operational Mode - Reports showing incident/resource summaries and recent audit history loaded from the running SQLite-backed application.

Operational Reports now presents persisted incident/resource summaries and audit history without exposing algorithm-run controls. Measured algorithm runs remain available under Academic / DSA Lab -> Efficiency Lab -> Recorded Runs, while the performance conclusions in Section 7 continue to use the dedicated full-efficiency-lab CSV evidence.

9. Responsible Algorithm Selection

The system does not treat one algorithm as universally best. Each technique is used where its assumptions and trade-offs are appropriate.

Table 9. Algorithm-selection rationale and limitations

Task	Selected approach	Why appropriate	When not appropriate
Urgent dispatch order	Custom priority heap	Efficient priority insert/extract and clear dispatch order	Not a substitute for exact budget optimization
Simple arrival-order dispatch	FIFO/circular/deque demonstrations	Models different service-order policies directly	Does not minimize weighted route cost
Reachability	BFS/DFS	Linear graph traversal and clear reachability evidence	Does not solve weighted shortest path
Weighted routing	Dijkstra	Finds shortest paths on non-negative weighted roads	Not valid for negative edge weights
Network spanning analysis	Prim/Kruskal	Produces minimum spanning network evidence	Not used as a point-to-point route algorithm
Resource recommendation	Greedy assignment	Fast, explainable local decision using eligible candidates	May not give a global optimum for every objective
Budget/capacity selection	Dynamic Programming	Exact optimum under manageable capacity state space	Pseudo-polynomial; may become expensive for huge capacities
Exact validation baseline	Brute Force	Simple exhaustive correctness reference for	Exponential growth makes it unsuitable for large n

		small n	
Search indexes	BST/RB Tree/B-Tree demos	Shows hierarchical indexing trade-offs	SQLite remains the persistent source of truth

10. Individual Contributions, Collaboration and Participation

The project used individual primary assignments together with cross-team integration. The contribution statements below reflect the team's recorded work allocation and supporting repository/assignment evidence where available. Git authorship is treated as supporting evidence, not as proof that one person alone designed every integrated component.

10.1 Cross-Level Collaboration and Knowledge Sharing

Level 200 members were primarily assigned algorithm implementations, but they were also required to learn the data structures directly related to those algorithms. They worked with the Level 300 members implementing the corresponding structures so that algorithm work was not isolated from the underlying representations. For example, members assigned BFS, DFS, Dijkstra, Prim and Kruskal studied and worked with the custom graph representations and supporting structures; members assigned optimization, searching, sorting and scheduling algorithms likewise learned the arrays, queues, heaps, trees and other structures on which those algorithms depend.

This arrangement supported knowledge transfer across the group and prepared members to explain both implementation details and the relationship between a data structure and the algorithm that uses it.

10.2 Individual Contributions

Index No.	Member	Level	Primary contribution
22079872	Isaac Morrison Quaye	Level 300	Group Leader. Dataset preparation and validation, SQLite database work, adjacency-list and adjacency-matrix graph representations, and overall project coordination.
22040957	Eastwood Tweneboah Osei	Level 300	Team Lead and GitHub Coordinator. Binary Search Tree, Red-Black Tree, shared interfaces, backend integration, repository coordination and final component integration.
22243032	Selorm Sem	Level 300	Team Lead and Frontend Lead. B-Tree, Hash Table, custom Set/Map structures, Swing frontend implementation, interface design and frontend consistency.
22153370	Fauziya Adjeley Adjei	Level 300	Custom Dynamic Array, Custom Linked List and Custom Iterator support, including associated structure behaviour and tests.
22136496	Nana Kwasi Agyiri	Level 300	Binary Heap, Priority Queue and priority-based dispatch functionality used to rank service requests.
22036173	Jephthah Peprah	Level 300	Stack, FIFO Queue, Circular Queue and Deque, including their use in workflow and scheduling demonstrations.
22404379	Abukari Issah Kangre	Level 200	Dynamic Programming and Brute-Force/Exhaustive Search for the request-optimization component.

Index No.	Member	Level	Primary contribution
22401243	Owusu Kenneth Kwabena	Level 200	Dijkstra shortest-path algorithm and Prim minimum-spanning-tree algorithm, including weighted-graph integration.
22392636	Asiedu Messiah	Level 200	Deputy and Algorithm Reviewer. Frontend support with Selorm; Linear Search, Binary Search, Selection Sort, Insertion Sort, algorithm traces and cross-team algorithm review.
22396052	Kumi Kwame Esua	Level 200	Greedy assignment, optimization examples and counterexamples demonstrating where greedy choices may fail to give a global optimum.
22409852	Owusu-Ansah John Nana Kwaku Bonsu	Level 200	Merge Sort and Quick Sort implementation and associated sorting functionality.
22367855	Amoh Derrick Kwaku	Level 200	Breadth-First Search and Depth-First Search for traversal of the campus graph.
22381560	Adjapong Stephanie Kyerewaa	Level 200	Disjoint-Set/Union-Find and Kruskal minimum-spanning-tree algorithm, including cycle detection during MST construction.
22325573	Jimoh Damilola Alliyah	Level 200	No contribution recorded.
22400447	Awuku Duke Asare	Level 200	No contribution recorded.

Table 10. Individual contribution summary

10.3 Meeting Attendance

The project schedule included eight team meetings on 24 July, 27 July, 31 July, 3 August, 7 August, 10 August, 14 August and 17 August 2026. Attendance is reported using each member's confirmed total across the eight meetings.

Index No.	Member	Meetings Attended	Attendance Rate
22079872	Isaac Morrison Quaye	8/8	100%
22040957	Eastwood Tweneboah Osei	8/8	100%
22243032	Selorm Sem	8/8	100%
22153370	Fauziya Adjeley Adjei	7/8	87.5%
22136496	Nana Kwasi Agyiri	8/8	100%
22036173	Jephthah Peprah	6/8	75%
22404379	Abukari Issah Kangre	6/8	75%
22401243	Owusu Kenneth Kwabena	5/8	62.5%
22392636	Asiedu Messiah	8/8	100%
22396052	Kumi Kwame Esua	6/8	75%
22409852	Owusu-Ansah John Nana Kwaku Bonsu	6/8	75%
22367855	Amoh Derrick Kwaku	6/8	75%
22381560	Adjapong Stephanie Kyerewaa	6/8	75%
22325573	Jimoh Damilola Alliyah	0/8	0%
22400447	Awuku Duke Asare	0/8	0%

Table 11. Individual attendance totals across eight project meetings

Across all 15 members, 88 of 120 possible meeting attendances were recorded (73.3%). Five members attended all eight meetings; two members had no recorded attendance.

10.4 Development Log

The development log was reconstructed from local Git history, refreshed remote-tracking metadata and authenticated GitHub pull-request metadata. The final repository history contains 40 pull requests: 39 merged and one closed without merge, with no open pull requests as of 25 August

2026. The milestone table summarizes the development period from 30 July to 25 August 2026; phase labels are not claims about exact individual working hours.

Date / period	Phase	Work completed	Evidence
2026-07-30	Repository and project setup	Created the repository, Java package skeleton, and initial README; later established the UG Campus Security Optimizer identity.	2a6b5c3, 918c305, eb57a99, bff6129
2026-08-04	Shared graph contracts	Added the WeightedGraph interface, normalized Edge, and graph-result contracts with unit tests.	95500bc, merged as PR #1
2026-08-05	Graph representations	Added deterministic adjacency-list and adjacency-matrix weighted graph implementations, contract tests, and build/prohibited-token evidence.	8ca002b, PR #2; 5eeb5bc, PR #8
2026-08-05–06	Search and elementary sorting	Implemented linear/binary search and selection/insertion sort, followed by package and review corrections.	81eea12, d1d76b8, 2f57b3f, 81ee55e; PRs #3–#6
2026-08-06–07	Advanced structures and sorting	Added hash table, custom map/set, B-tree, binary heap, merge sort, and quick sort with tests or subsequent fixes.	ad76d03, 0d69c2d, b557f62, e0fd912, 486862d; PRs #9, #10, #14, #16
2026-08-07	Ordered trees and traversal algorithms	Added BST/red-black-tree implementations and initial BFS/DFS traversal algorithms with tests.	281eaf5, bf1e73e; PRs #11 and #13
2026-08-07	Maven build alignment	Migrated the repository from a plain javac arrangement to Maven/JUnit conventions.	c298ad3
2026-08-08	UG-localised dataset and evidence	Added the seven CSV data files, data dictionary, provenance, validation report, manifest, and dataset screenshots.	ba0ff57, PR #12; follow-up 57b2c1b, PR #15
2026-08-08–10	Review fixes and linear/scheduling structures	Addressed BFS/DFS review issues; added dynamic array, linked list, iterator, stack, FIFO queue, circular queue, and deque with tests.	a42791b, PR #18; fe09f1c, PR #19; a39d516, PR #17
2026-08-09–11	SQLite database foundation and reload	Added schema/manager, transactional CSV importer/parser, DAO/mapper layer, and database-to-custom-graph loader; later merged the database branch.	2e76534, 0dfe231, 5a99ec1, 3e5c141; PR #20 (f8748d9)
2026-08-10–14	MSTs, project parameters, and optimisation integration	Added Dijkstra/Prim, disjoint set/Kruskal, index-derived parameters, and production DP/brute-force integration.	4593355, PR #26; a868a08, PR #25; 5dd0ad1, PR #23; 5e5ce6b, PR #27
2026-08-14	Backend workflow expansion	Added assignments, undo, audit/reporting, scenario loading, request history, and end-to-end service capability; the history also records a revert and reapplication of the backend-foundation merge.	514d401, PR #34; PRs #29–#33
2026-08-17–18	Backend finalisation and frontend compatibility	Finalised application bootstrap, end-to-end workflow checks, performance validation, and frontend-service compatibility over the real backend.	d6ef89a, PR #35; af370c8, PR #36; f50b81f
2026-08-18–20	Swing GUI integration	Merged the GUI foundation and connected optimisation/reporting screens to backend services; fixed data/display integration issues.	PR #28; 3bd83c7, PR #37 / dc0b1ad
2026-08-21	Release and UI correctness cleanup	Updated README runtime guidance, required explicit	ae840a2, ed9d367, 0707892, 4b75bac

Date / period	Phase	Work completed	Evidence
		location metadata, moved UI actions off the event thread, and exposed routing scenarios and assignment details.	
2026-08-21	Examiner-facing DSA evidence	Added scheduling/index demonstrations, correctness-trace generation, formal operation/correctness/complexity notes, and the Swing efficiency lab.	56bb286
2026-08-21	Benchmark methodology refinement	Strengthened the efficiency-lab methodology and representative benchmark evidence.	9299792
2026-08-21	Full performance evaluation	Executed the complete official-size Efficiency Lab using benchmark seed 7497, producing 336 raw trial rows, an averaged summary, environment metadata and nine report graphs.	results/full-efficiency-lab/ benchmark artefacts (local evidence)
2026-08-25	Operational / academic UI separation	Added the final two-mode Swing interface: Operational Mode (Dashboard, Incidents, Dispatch, Routes, Resources, Reports) and Academic / DSA Lab (Structures, Search & Sort, Graph Algorithms, Optimization, Correctness, Efficiency Lab); added user-facing formatting and focused UI tests.	5773b29; PR #40; main ff7f47a; 681 tests

Table 12. Evidence-based development milestones from Git and GitHub history

The repository began with a Java project skeleton and shared graph contracts, then moved through feature-branch implementation of the assessed custom structures and algorithm families. Search/sort work, graph representations, ordered/hash structures, queues, traversal, shortest-path and MST components appear as distinct development milestones rather than a single late-stage code drop.

The University of Ghana-localized dataset was followed by the SQLite schema, transactional import, DAO/mapper layer and graph reload support. Backend workflow work then connected request status, assignments, routing, undo/audit and reporting before the Swing frontend was integrated with the real services.

The final development phase focused on correctness evidence, reproducible performance analysis, and separation of the operator-facing workflow from the Academic / DSA Lab. The Git audit records iterative review and correction, while the full local benchmark run adds the final 336-trial empirical evidence used in Section 7.

10.5 AI-Assistance Disclosure

AI tools were used in a supporting role, including repository and compliance auditing, difficult debugging and integration diagnosis, and reconstruction of the development history from Git/GitHub evidence. The team reviewed and validated resulting suggestions against the repository, tests, application behaviour and project evidence, and remains responsible for the system design, implementation decisions, verification, interpretation of results and oral defence.

11. References

1. University of Ghana, Department of Computer Science. DCIT 204/308: Data Structures and Algorithms I & II - Joint DSA Semester Project: Ghana Smart Service Operations Optimizer. Official project brief supplied for the semester project.

2. University of Ghana, Department of Computer Science. DCIT 204/308 Project Submission Checklist and Cover Sheet.
3. Project repository documentation: README, data dictionary, dataset provenance, correctness evidence and performance methodology files.
4. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. Introduction to Algorithms. (Listed as a suggested reference in the official brief.)
5. Sedgewick, R., and Wayne, K. Algorithms. (Listed as a suggested reference in the official brief.)
6. Goodrich, M. T., Tamassia, R., and Goldwasser, M. H. Data Structures and Algorithms in Java. (Listed as a suggested reference in the official brief.)

12. Appendices

Appendix A - Index Numbers and Parameter Derivation

Index numbers used by the current project documentation: 22079872, 22040957, 22243032, 22153370, 22136496, 22036173, 22404379, 22401243, 22392636, 22396052, 22409852, 22367855, 22381560, 22325573, 22400447.

Sum of final two digits = 897. Priority weight = 1 + (897 mod 5) = 3. Optimization budget = 50 + (897 mod 51) = 80. Sum of final three digits = 7497, used as the deterministic benchmark seed.

Appendix B - Benchmark Evidence Inventory

Table 13. Final efficiency-lab artefacts included with the project

Artifact	Purpose
benchmark-results.csv	336 raw trial rows from the full official-size benchmark run
benchmark-summary.csv	Averaged report-ready benchmark groups
environment.txt	Seed, OS, JVM, processor count and memory-method metadata
01_search_runtime.png	Linear versus Binary Search
02_sorting_runtime.png	Selection/Insertion/Merge/Quick Sort
03_hash_collisions.png	Hash collision/load condition comparison
04_tree_search_runtime.png	BST versus Red-Black Tree search
05_tree_height.png	BST versus Red-Black Tree height
06_heap_runtime.png	Heap insert/extract scaling
07_graph_runtime.png	BFS/DFS/Dijkstra/Prim/Kruskal
08_optimization_runtime.png	DP versus Brute Force
09_greedy_assignment_runtime.png	Greedy Assignment scaling

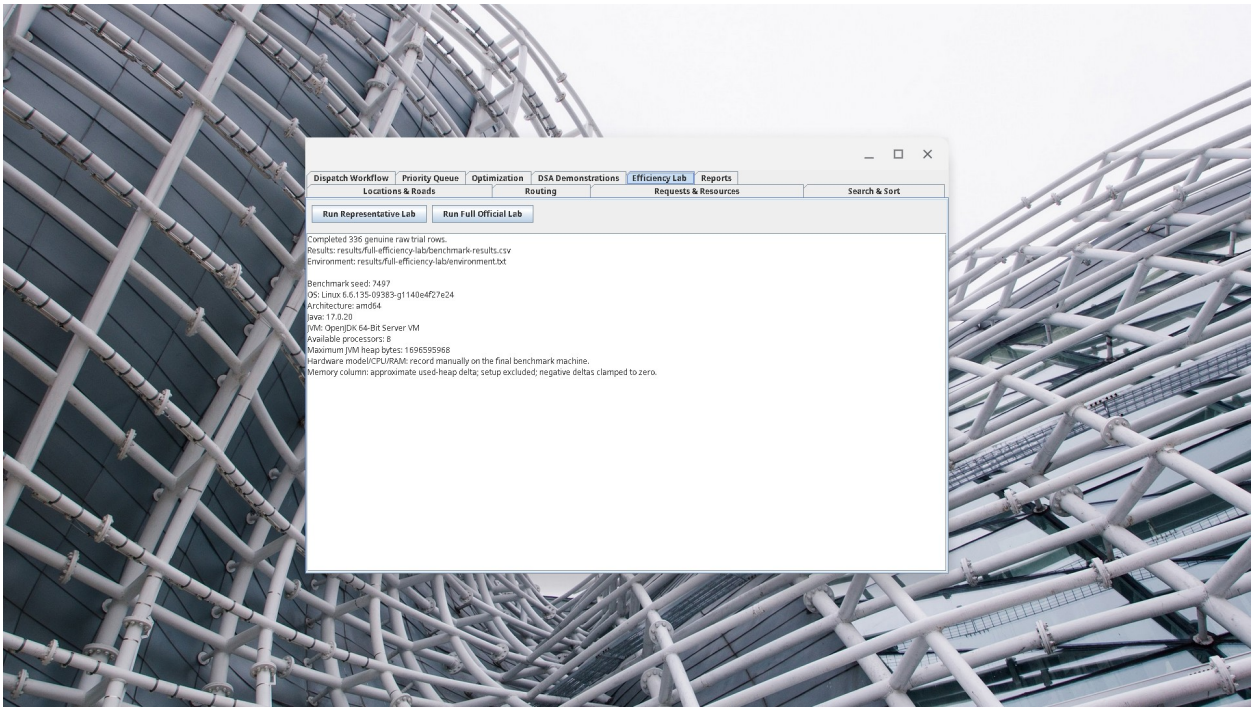

Figure 21. Academic / DSA Lab - Efficiency Lab evidence from the completed full official benchmark suite.

Appendix C - Submission Checklist Status

Table 14. Final submission checklist status

Checklist item	Current status
Local dataset with data dictionary	Complete
Database schema and seed data	Complete
Custom data structures	Complete
Searching and sorting algorithms	Complete
Graph algorithms	Complete
Greedy and DP algorithms	Complete
Correctness tests and trace tables	Complete; readable trace/proof/counterexample evidence included
Performance CSV and graphs	Complete
Technical report	Complete; final DOCX/PDF updated for Operational Mode and Academic / DSA Lab, including screenshots, benchmarks, contributions, attendance and development log.
Demo video / oral defense	Separate submission/defense preparation still required

Appendix D - AI Assistance Evidence

This appendix provides concise supporting evidence for representative AI-assisted activities used during the project. It is not a complete chat transcript. AI output was treated as advisory: repository evidence, tests, application behaviour and team review remained the basis for accepting technical changes or claims.

D.1 Repository Compliance Audit

Purpose. To compare the implemented system against the official DCIT 204/308 project requirements and identify genuine remaining gaps.

Representative supporting prompt:

> *Audit the University of Ghana Campus Security and Emergency Response Optimizer repository against the official DCIT 204/308 project brief. Verify the local dataset and database, required custom data structures, searching and sorting algorithms, graph algorithms, greedy and dynamic-programming components, correctness evidence, performance experiments, examiner demonstrations and submission requirements. Be strict. Do not modify files, commit, merge or push. Distinguish completed requirements from genuine remaining gaps and cite repository evidence for each conclusion.*

How it was used. The audit was used as a checklist. Reported issues were checked against the repository and official brief before any change or compliance claim was accepted.

D.2 Difficult Debugging and Integration

Purpose. To assist with diagnosis of complex interactions between the Swing interface, application services, routing, optimization and persistence components.

Representative supporting prompt:

> *Inspect the current integrated Java/Maven application and diagnose the frontend/backend integration problem without replacing the assessed custom data structures or algorithms. Trace the failure from the Swing action through the service layer to the underlying algorithm or database operation. Preserve existing public contracts where possible, add a regression test for any confirmed defect, and verify the Maven test suite after the fix.*

How it was used. Suggested diagnoses and fixes were reviewed against the existing architecture and retained only after the affected behaviour and regression tests were verified.

D.3 Development-History Reconstruction

Purpose. To reconstruct the development log from Git and GitHub evidence instead of relying on memory.

Representative supporting prompt:

> *Reconstruct a factual development log for the University of Ghana Campus Security and Emergency Response Optimizer directly from Git and GitHub history. Inspect branches, commits, merge history, pull requests, authors and important diffs. Do not modify source code, create commits, merge or push. Produce a concise chronological development log and clearly distinguish repository-supported facts from attribution that cannot be independently verified.*

How it was used. The resulting history audit inspected 108 reachable main-branch commits and 40 pull requests (39 merged and one closed without merge). The reconstructed milestones were reviewed before being summarized in the development-log section of this report.

D.4 Verification and Responsibility

AI-generated suggestions were not treated as authoritative project results. Code changes were verified through the project test suite and application demonstrations. Performance conclusions were based on the team's own benchmark execution and exported CSV evidence. The team remains responsible for understanding, explaining and modifying the submitted implementation.

D.5 Supporting Evidence

The primary evidence for the submitted work remains the Git repository and pull-request history, source code, JUnit and integration tests, SQLite/CSV data, generated correctness traces, benchmark CSV files, performance graphs, and demonstrations from both Operational Mode and Academic / DSA Lab. Additional prompt records can be supplied if requested during assessment.